

UCUENCA

INGENIERÍA

MAESTRÍA EN CIENCIAS DE LA INGENIERÍA ELÉCTRICA

PERÍODO 202664-PA01 · MAR 2026 – AGO 2026

Manual de Prácticas Robótica

Ing. Henry Maldonado, Ing. Galo Guzmán

Departamento de Ingeniería Eléctrica, Electrónica y Telecomunicaciones

Cuenca – Ecuador 2026

Índice

1	Introducción	3
1.1	Entorno de trabajo	3
1.1.1	QUBE Servo 3	3
1.1.2	Helicóptero de 1 DOF	8
1.2	Modelado de sistemas	12
1.2.1	Modelado del motor DC (QUBE-Servo 3)	12
1.2.2	Modelado del helicóptero de un grado de libertad	16
1.3	Controladores	18
1.3.1	Análisis del Control Proporcional (P)	18
1.3.2	Análisis del Control Proporcional-Integral (PI)	19
1.3.3	Análisis del Control Proporcional-Integral-Derivativo (PID)	21
2	Práctica 1: Control de velocidad con el Qube Servo 3	22
2.1	Introducción	22
2.2	Objetivos de aprendizaje	23
2.3	Materiales requeridos	23
2.4	Procedimiento	24
2.4.1	Control de velocidad a lazo abierto	24
2.4.2	Control de velocidad a lazo cerrado	28
2.5	Reto	34
2.6	Preguntas de análisis	35
2.7	Entregables	35
2.8	Rúbrica de Evaluación	36
3	Práctica 2: Control de posición con el Qube Servo 3	37
3.1	Introducción	37
3.2	Objetivos de aprendizaje	38
3.3	Materiales requeridos	38
3.4	Procedimiento	39
3.4.1	Control de posición a lazo abierto	39
3.4.2	Control de posición a lazo cerrado	43
3.5	Reto	48
3.6	Preguntas de análisis	49
3.7	Entregables	49
3.8	Rúbrica de Evaluación	50
4	Práctica 3: Controlador de posición para helicóptero de un grado de libertad	51
4.1	Introducción	51
4.2	Objetivos de aprendizaje	51
4.3	Materiales requeridos	52
4.4	Procedimiento	52
4.4.1	Controlador PID en zona local	52

4.5	Reto: Control PID por ganancia programable (<i>Gain Scheduling</i>)	56
4.6	Preguntas de análisis	56
4.7	Entregables	57
4.8	Rúbrica de Evaluación	57

Capítulo 1

Introducción

1.1 Entorno de trabajo

1.1.1 QUBE Servo 3

El QUBE-Servo 3 es un sistema electromecánico diseñado para realizar prácticas de control automático en tiempo real. El equipo consta de un motor de corriente continua (DC) acoplado directamente a sensores (encoders) que permiten medir con precisión la posición y la velocidad del eje.

Su característica principal es su diseño modular. Dependiendo de la práctica a realizar, se pueden montar distintos accesorios sobre el motor:

- **Disco de inercia:** Para estudiar sistemas lineales y control de velocidad o posición.
- **Péndulo rotatorio:** Para experimentos de equilibrio y sistemas no lineales más complejos.



Figura 1.1: QUBE-Servo 3 de Quanser. Fuente: [1]

El sistema se conecta a la computadora para programar leyes de control y ver cómo responde el hardware físico en comparación con la simulación. A continuación, se comparten recursos adicionales para profundizar en el conocimiento del sistema y su integración con herramientas de control como Simulink:

- [Official website](#) .
- [Info Sheet](#) .
- [Research Resources](#) .

- [Qube-Servo 3 Experiment](#) ↗ .
- [Documentación adicional](#) ↗ .

La Tabla 1.1 presenta los principales parámetros físicos y eléctricos del sistema QUBE-Servo 3, incluyendo características del motor de corriente continua, módulos mecánicos acoplables, sensores de medición y especificaciones del amplificador.

Cuadro 1.1: Parámetros del sistema QUBE-Servo 3 (adapted from [2])

Symbol	Description	Value
V_{nom}	Nominal input voltage	24.0 V
τ_{nom}	Nominal torque	20.4 mN·m
ω_{nom}	Nominal speed	5400 RPM
I_{nom}	Nominal current	0.5 A
R_m	Terminal resistance	7.5 Ω
k_t	Torque constant	0.0422 N·m/A
k_m	Motor back-EMF constant	0.0422 V/(rad/s)
J_m	Rotor inertia	1.4×10^{-6} kg·m ²
L_m	Rotor inductance	1.15 mH
m_h	Module attachment hub mass	0.0106 kg
r_h	Module attachment hub radius	0.0111 m
J_h	Module attachment moment of inertia	0.6×10^{-6} kg·m ²
Inertia Disc Module		
m_d	Disc mass	0.053 kg
r_d	Disc radius	0.0248 m
Rotary Pendulum Module		
m_r	Rotary arm mass	0.095 kg
L_r	Rotary arm length (pivot to end of metal rod)	0.085 m
m_p	Pendulum link mass	0.024 kg
L_p	Pendulum link length	0.129 m
Motor and Pendulum Encoders		
–	Encoder line count	512 lines/rev
–	Encoder line count in quadrature	2048 lines/rev
–	Encoder resolution (quadrature, deg)	0.176 deg/count
–	Encoder resolution (quadrature, rad)	0.00307 rad/count
Amplifier		
–	Amplifier type	PWM
–	Peak stall current	1.8 A @ 15V
–	Continuous sustainable current	0.5 A @ 5V
–	Output voltage range (recommended)	±10 V
–	Output voltage range (maximum)	±15 V
–	Default deadband	0.65 V

Conexión básica con Simulink

1. Abrir Simulink y crear un nuevo modelo QUBE-Servo3 (QUARC Win64) (ver Figura 1.2).

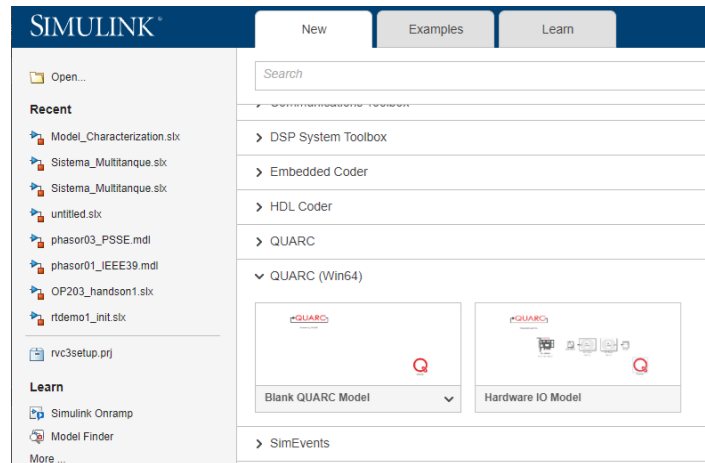


Figura 1.2: Creación de un nuevo modelo en Simulink para el sistema QUBE-Servo3.

2. Añadir el bloque **HIL Initialize** y dando doble clic, configurar los parámetros de acuerdo al sistema Qube Servo 3 como se muestra en la Figura 1.3.

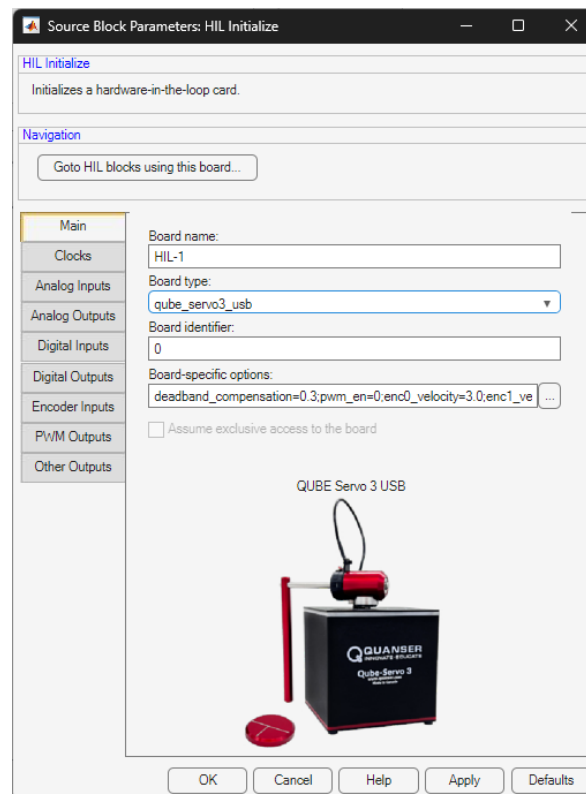


Figura 1.3: Configuración del bloque HIL Initialize para el sistema Qube Servo 3.

3. Añadir el bloque **HIL Write**, que por defecto viene configurado para el canal 0, que corresponde al voltaje de entrada del Qube Servo 3.

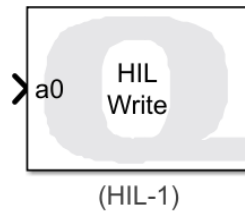


Figura 1.4: Bloque HIL Write para enviar señales de control al Qube Servo 3.

4. Añadir el bloque **HIL Read Timebase**, que por defecto viene configurado para leer el canal analógico 0, pero para obtener la velocidad angular, se puede utilizar la salida *Other channels* en el canal 14000 y para la lectura de la posición angular, se puede la salida de *Encoder channels* en el canal 0, como se muestra en la Figura 1.6.

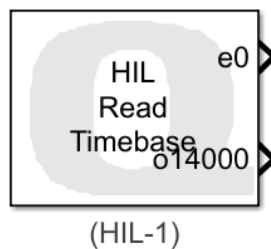


Figura 1.5: Bloque HIL Read para adquirir datos de posición angular del Qube Servo 3.

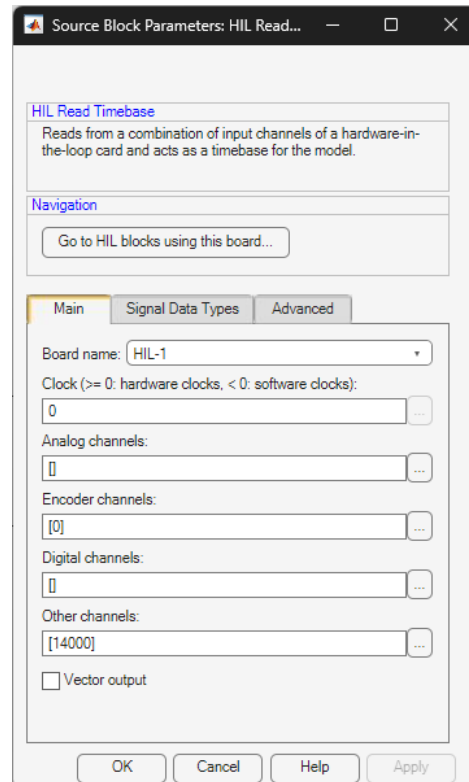


Figura 1.6: Configuración del bloque HIL Read Timebase para el encoder del QUBE-Servo 3.

5. La configuración básica necesaria en Simulink para la adquisición de datos y control del Qube Servo 3 se muestra en la Figura 1.7.

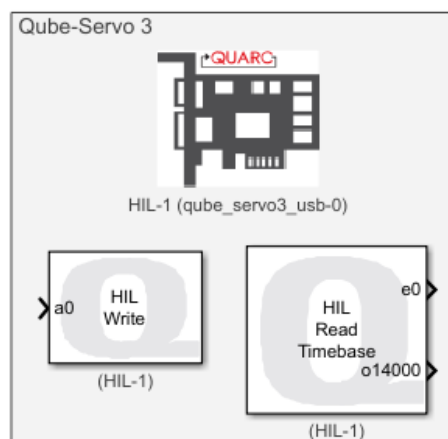


Figura 1.7: Configuración básica del modelo en Simulink para el QUBE-Servo 3.

1.1.2 Helicóptero de 1 DOF

El helicóptero de un grado de libertad (Figura 1.8) es un sistema no lineal: un brazo rígido rota en el plano XZ alrededor de un eje fijo Y . En un extremo se ubica el motor con hélice, que genera una fuerza de empuje tangencial, y en el otro extremo la cola; además existe un contrapeso perpendicular al brazo principal para equilibrar el sistema.

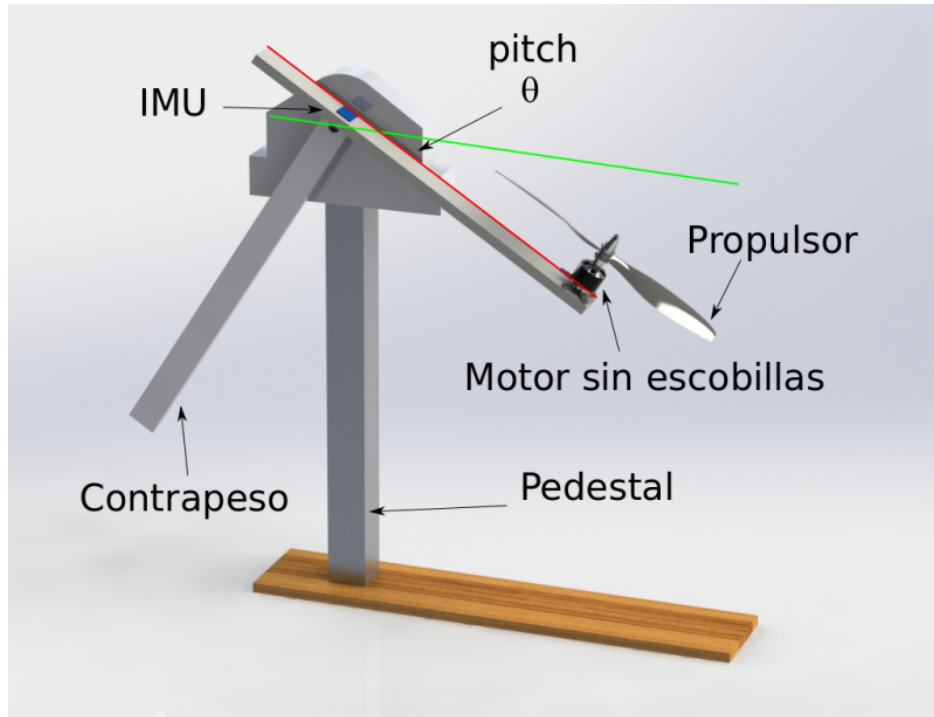


Figura 1.8: Partes del helicóptero de un grado de libertad [3].

Arquitectura general

El funcionamiento del sistema se organiza en tres niveles jerárquicos que interactúan de forma bidireccional para lograr el control del helicóptero (ver Figura 1.9).

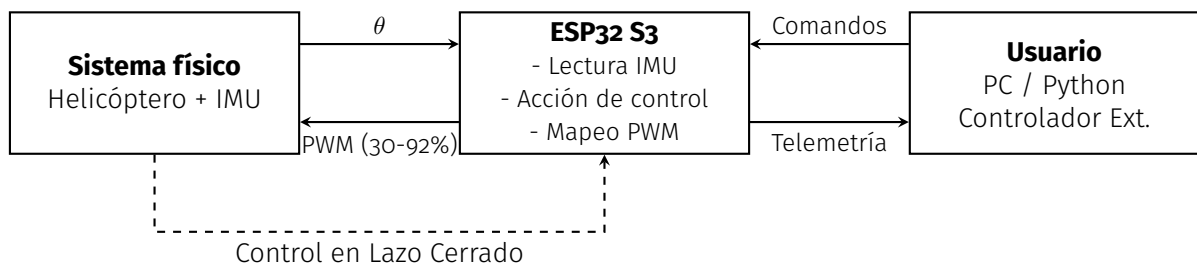


Figura 1.9: Arquitectura de control con comunicación bidireccional.

- Nivel de usuario: Representado por una interfaz en Python corriendo en una PC. Su función es la supervisión y el envío de parámetros de alto nivel (setpoints, ganancias, modos de operación) mediante un protocolo de comunicación serial.

- Nivel de control (ESP32): Es el núcleo del sistema. Aquí reside la lógica de tiempo real que integra la lectura del sensor IMU, el cálculo del algoritmo PID y la generación de la señal PWM. Este nivel garantiza que el período de muestreo se mantenga constante.
- Nivel físico (Planta): Comprende el hardware mecánico y electrónico de potencia. El ESC traduce el PWM en velocidad de motor, generando el empuje que altera la posición angular, la cual es cerrada en lazo mediante la retroalimentación de la IMU.

Conexión de hardware

El sistema está integrado por los dispositivos detallados en la Tabla 1.2. En esta se especifican los componentes de hardware utilizados y su función principal dentro del montaje físico.

Cuadro 1.2: Resumen de la Configuración del Sistema

Componente	Hardware	Responsabilidad física
Microcontrolador	ESP32 S3	- Procesamiento central - Gestión de señales I/O
Sensor	IMU GY-86	Captura de datos inerciales
Actuador	ESC	Interfaz de potencia para el motor
Propulsión	Motor y Hélice	Generación de empuje mecánico

El esquema de conexiones se presenta en la Figura 1.10. La IMU se comunica con el ESP32 mediante el bus I2C (pines GPIO 8 y 9), mientras que el ESC recibe la señal de control desde el pin GPIO 4. Para la monitorización de datos, el envío de comandos y controladores externos, se utiliza un enlace serial entre el microcontrolador y un computador externo.

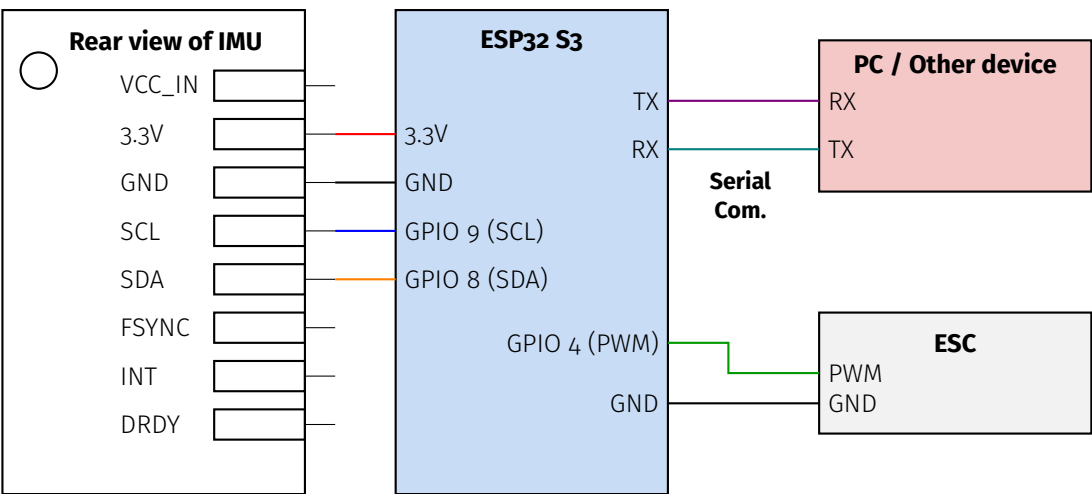


Figura 1.10: Esquema de conexión.

Firmware - Estados de operación

Para garantizar la seguridad del sistema físico, el firmware implementa tres estados principales:

- **IDLE (Seguro):** Estado inicial por defecto. El motor permanece apagado (Potencia al 0% $\approx$ 30% Duty Cycle). El sistema solo sale de este estado tras recibir un comando válido de control o manual.
- **MANUAL:** El usuario define directamente el porcentaje de potencia aplicado al motor (0.0 - 100.0). Este modo permite tanto experimentos a lazo abierto, identificación del sistema y un controlador externo.
- **CONTROL INTERNO:** El lazo cerrado está activo. El ESP32 recibe la referencia de posición angular mediante comunicación serial. Internamente, implementa un controlador PID Gain Scheduling que ajusta sus ganancias en función del setpoint.

Protocolo de comunicación serial (PC → ESP32)

La interacción con el sistema se realiza mediante comunicación serial con cualquier dispositivo que lo soporte. En la Tabla 1.3 se detallan los comandos disponibles para gestionar el sistema.

Comando	Descripción	Ejemplo
M:<pct>	Activa el modo MANUAL . El parámetro pct [0-100] define la potencia del motor.	M:45.5
CIN:<deg>	Activa el modo CONTROL INTERNO . El parámetro deg [0-45] define el setpoint angular.	CIN:30
FREQ:<ms>	Ajusta el período de muestreo del sistema en milisegundos [10-500].	FREQ:100
CAL	Inicia la rutina de calibración del ESC (Solo ejecutable una vez por encendido).	CAL
I	Comando de emergencia. Regresa el sistema al estado IDLE de inmediato.	I

Cuadro 1.3: Protocolo de comandos seriales.

Formato de telemetría (ESP32 → PC)

El sistema envía una trama de datos separada por comas (formato CSV) en cada ciclo de muestreo para facilitar su procesamiento en Python:

<tiempo_ms>, <angulo_deg>, <potencia_pct>, <modo_id>

Donde modo_id es un valor entero: 0 (IDLE), 1 (MANUAL), 2 (CONTROL INTERNO).

Mapeo del rango operativo

Debido a que los controladores de velocidad (ESC) comerciales suelen ignorar señales de PWM muy bajas por seguridad, el firmware traduce el porcentaje solicitado por el usuario [0–100%] a un rango de señal real compatible con el hardware [30–95%], como se observa en la Figura 1.11.

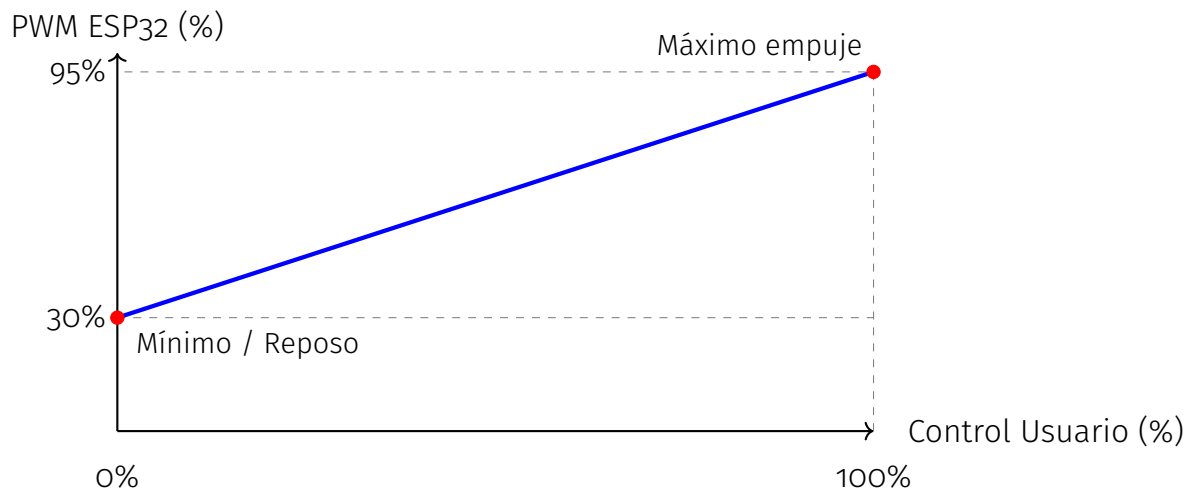


Figura 1.11: Mapeo entre el nivel de control y el ciclo de trabajo del PWM.

⚠ Importancia de la Calibración

- El comando **CAL** es obligatorio al energizar el sistema por primera vez. Envía las señales de 95% y 30% de DC de forma secuencial para que el ESC reconozca los límites mínimo y máximo de operación.
- **Advertencia:** Durante la calibración, el motor puede emitir pitidos; no manipule la hélice durante este proceso.
- **Advertencia:** Una vez calibrado, el motor reconocerá cualquier valor entre el 30% y el 95% del PWM del ESP32 S3 como un rango válido y operará, por lo que no se debe repetir el proceso a menos que se haya desenergizado el sistema.

1.2 Modelado de sistemas

1.2.1 Modelado del motor DC (QUBE-Servo 3)

Planteamiento electromecánico

El sistema del motor DC puede modelarse a partir de sus componentes eléctricos y mecánicos, considerando el acoplamiento electromecánico, como se presenta en la Figura 1.12.

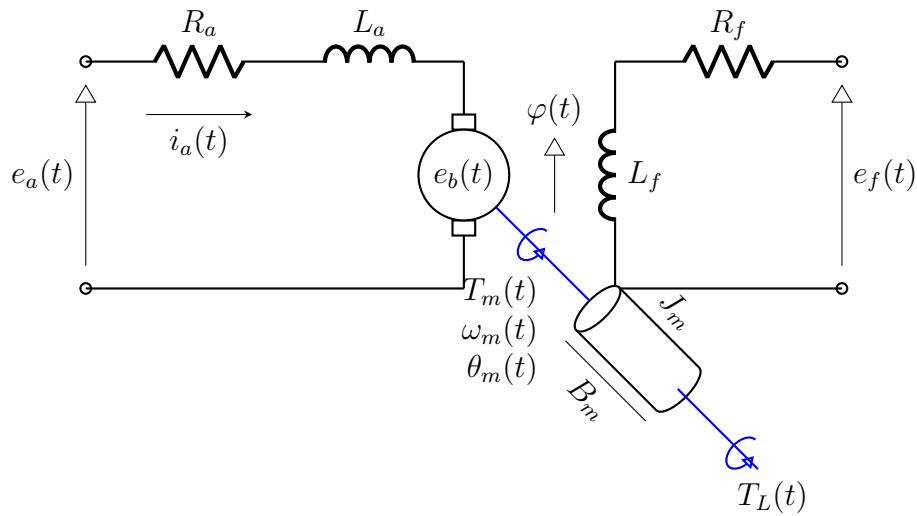


Figura 1.12: Diagrama esquemático electromecánico del motor de corriente continua.

Consideraciones previas: para el siguiente desarrollo matemático, se asume que el flujo magnético generado por el estator es constante.

Modelo eléctrico

Aplicando la Ley de Voltajes de Kirchhoff al circuito de armadura:

$$e_a(t) = R i_a(t) + L \frac{di_a(t)}{dt} + e_b(t) \quad (1.1)$$

donde:

- $e_a(t)$ es el voltaje de entrada
- R es la resistencia de armadura
- L es la inductancia
- $i_a(t)$ es la corriente
- $e_b(t)$ es la fuerza contraelectromotriz

La fuerza contraelectromotriz se define como:

$$e_b(t) = K_v \omega(t) \quad (1.2)$$

Modelo mecánico

Aplicando la segunda ley de Newton para rotación:

$$T_m(t) = J_m \frac{d\omega(t)}{dt} + b_m \omega(t) \quad (1.3)$$

donde:

- $T_m(t)$ es el torque del motor
- J_m es el momento de inercia
- b_m es el coeficiente de fricción viscosa
- $\omega(t)$ es la velocidad angular

El torque del motor está relacionado con la corriente:

$$T_m(t) = K_t i_a(t) \quad (1.4)$$

Transformada de Laplace

Aplicando transformada de Laplace (condiciones iniciales nulas):

$$E_a(s) = (R + Ls)I_a(s) + E_b(s) \quad (1.5)$$

$$E_b(s) = K_v \Omega(s) \quad (1.6)$$

$$T_m(s) = (J_m s + b_m) \Omega(s) \quad (1.7)$$

$$T_m(s) = K_t I_a(s) \quad (1.8)$$

Relación entrada-salida

Sustituyendo:

$$I_a(s) = \frac{E_a(s) - K_v \Omega(s)}{Ls + R} \quad (1.9)$$

Multiplicando por K_t :

$$K_t I_a(s) = \frac{K_t(E_a(s) - K_v \Omega(s))}{Ls + R} \quad (1.10)$$

Igualando con el modelo mecánico:

$$\frac{K_t(E_a(s) - K_v \Omega(s))}{Ls + R} = (J_m s + b_m) \Omega(s) \quad (1.11)$$

Multiplicando ambos lados por $(Ls + R)$:

$$K_t E_a(s) - K_t K_v \Omega(s) = (J_m s + b_m)(Ls + R) \Omega(s) \quad (1.12)$$

Agrupando términos:

$$K_t E_a(s) = [(J_m s + b_m)(Ls + R) + K_t K_v] \Omega(s) \quad (1.13)$$

Función de transferencia teórica

Finalmente, la función de transferencia del sistema es:

$$\frac{\Omega(s)}{E_a(s)} = \frac{K_t}{(J_m s + b_m)(Ls + R) + K_t K_v} \quad (1.14)$$

Parametrización para el QUBE-Servo 3 con disco de inercia

A partir de la hoja de datos del QUBE-Servo 3, se sustituyen los valores físicos en la función de transferencia teórica.

Dado que el sistema opera con el módulo de disco montado, el momento de inercia equivalente es:

$$J_{eq} = J_m + J_h + J_d \quad (1.15)$$

El momento de inercia del disco se aproxima como cilindro sólido ($J = \frac{1}{2}mr^2$):

$$J_d = \frac{1}{2}m_d r_d^2 = \frac{1}{2}(0.053 \text{ kg})(0.0248 \text{ m})^2 \approx 16.3 \times 10^{-6} \text{ kg} \cdot \text{m}^2 \quad (1.16)$$

Por lo tanto:

$$J_{eq} = (1.4 \times 10^{-6}) + (0.6 \times 10^{-6}) + (16.3 \times 10^{-6}) = 18.3 \times 10^{-6} \text{ kg} \cdot \text{m}^2 \quad (1.17)$$

Equivalencia de parámetros:

- Resistencia de armadura (R): $R_m = 7.5 \Omega$
- Inductancia (L): $L_m = 1.15 \text{ mH} = 1.15 \times 10^{-3} \text{ H}$
- Constante de torque (K_t): $k_t = 0.0422 \text{ N} \cdot \text{m/A}$
- Constante electromotriz (K_v): $k_m = 0.0422 \text{ V/(rad/s)}$

Nota sobre la fricción viscosa (b_m): este parámetro no se especifica en los valores nominales y suele ser despreciable para análisis de primer orden ($b_m \approx 0$).

Sustituyendo ($J_m \rightarrow J_{eq}$), con $b_m = 0$:

$$\frac{\Omega(s)}{E_a(s)} = \frac{0.0422}{(18.3 \times 10^{-6} s)(1.15 \times 10^{-3} s + 7.5) + 0.00178} \quad (1.18)$$

A partir de la ecuación característica del denominador:

- **Frecuencia natural:** $\omega_n = \sqrt{\frac{0.00178}{2.1045 \times 10^{-8}}} \approx 290.90 \text{ rad/s} \approx 46.30 \text{ Hz}$
- **Factor de amortiguamiento:** $\zeta = \frac{1.3725 \times 10^{-4}}{2\sqrt{(2.1045 \times 10^{-8})(0.00178)}} \approx 11.21$

Modelado práctico a partir de medición en lazo abierto

Para las prácticas también se usa una aproximación experimental basada en las señales medidas del encoder/tacómetro.

Conversión de posición angular: Usando un bloque *HIL Read Timebase* agregando una salida *a0*, devolverá la posición en pulsos. La conversión a radianes se realiza mediante:

$$\theta_m(t) = \frac{2\pi \cdot P}{N} \quad (1.19)$$

Conversión de velocidad angular: Usando un bloque *HIL Read Timebase* agregando una salida *o14000*, devolverá la velocidad en pulsos por segundo. La conversión a radianes por segundo se realiza mediante:

$$\omega_m(t) = \frac{2\pi \cdot P}{N} \quad (1.20)$$

donde N es el número de pulsos por revolución y P corresponde a pulsos (posición) o pulsos por segundo (velocidad).

Con base en pruebas a lazo abierto, se emplean las funciones de transferencia aproximadas:

$$\frac{\Omega_m(s)}{V_m(s)} = \frac{K}{\tau s + 1} \quad (1.21)$$

$$\frac{\Theta_m(s)}{V_m(s)} = \frac{K}{s(\tau s + 1)} \quad (1.22)$$

1.2.2 Modelado del helicóptero de un grado de libertad

Modelo dinámico del sistema

El helicóptero puede modelarse como un sistema rotacional alrededor del eje Y , con variable de interés el ángulo θ en el plano XZ , como se muestra en la Figura 1.13.

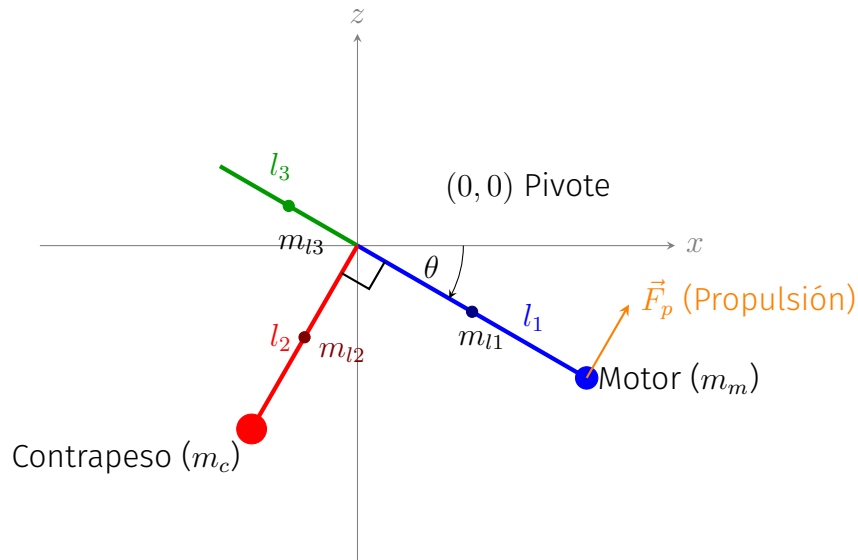


Figura 1.13: Esquema del sólido rígido en el plano XZ .

Elementos

- m_m : masa del motor en el extremo del brazo l_1 .
- m_{l1} : masa distribuida del brazo l_1 .
- m_c : masa del contrapeso en el extremo del brazo l_2 .
- m_{l2} : masa distribuida del brazo l_2 .
- m_{l3} : masa distribuida del brazo de la cola l_3 .

El comportamiento dinámico queda determinado por:

- **Torque motor (τ_{motor})**: generado por la fuerza de empuje del motor.
- **Torque gravitacional (τ_{grav})**: producido por los pesos de los elementos del sistema.
- **Torque de fricción ($\tau_{friccion}$)**: asociado a pérdidas mecánicas en el eje.

Ecuación de movimiento

Aplicando la segunda ley de Newton para rotación:

$$\sum \tau = I_{total} \ddot{\theta} \quad (1.23)$$

Considerando los torques presentes:

$$I_{total} \ddot{\theta} = \tau_{motor} - \tau_{grav}(\theta) - b\dot{\theta} \quad (1.24)$$

Modelo no lineal del sistema

La dependencia trigonométrica del torque gravitacional con respecto al ángulo θ introduce no linealidad. La dinámica completa se expresa como:

$$I_{total} \ddot{\theta} + b\dot{\theta} + g[(A - B) \cos \theta + C \sin \theta] = F_p \cdot l_1 \quad (1.25)$$

Consideraciones para laboratorio

- En la práctica no se dispone de todos los parámetros físicos exactos del sistema (masas, inercias y coeficientes), por lo que este modelo se utiliza como referencia conceptual.
- El diseño final del controlador se realiza de forma experimental alrededor de un punto de operación.

1.3 Controladores

1.3.1 Análisis del Control Proporcional (P)

El control proporcional es la estrategia de lazo cerrado más simple, donde la señal de control $u(t)$ es directamente proporcional al error del sistema $e(t)$ [4]. El esquema general se ilustra en la Figura 1.14.

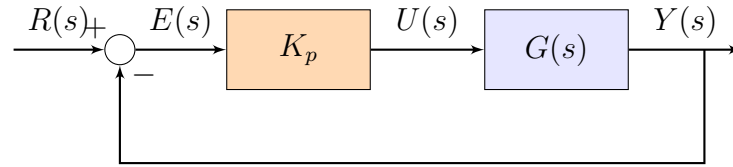


Figura 1.14: Diagrama de bloques de un sistema de control de lazo cerrado con controlador Proporcional.

Para analizar el efecto del controlador, se asume una planta genérica de primer orden definida por $G(s) = \frac{K}{\tau s + 1}$, donde K es la ganancia de la planta y τ es su constante de tiempo. La función de transferencia del sistema en lazo cerrado con un controlador $C(s) = K_p$ y realimentación unitaria es:

$$G_{lc}(s) = \frac{C(s)G(s)}{1 + C(s)G(s)} = \frac{K_p \left(\frac{K}{\tau s + 1} \right)}{1 + K_p \left(\frac{K}{\tau s + 1} \right)} = \frac{K_p K}{\tau s + 1 + K_p K} \quad (1.26)$$

Dividiendo el numerador y el denominador por el término $(1 + K_p K)$, la función se reescribe en su forma estándar de primer orden:

$$G_{lc}(s) = \frac{\left(\frac{K_p K}{1 + K_p K} \right)}{\left(\frac{\tau}{1 + K_p K} \right) s + 1} = \frac{K_{lc}}{\tau_{lc} s + 1} \quad (1.27)$$

De este análisis se extraen dos características fundamentales que definen al control proporcional:

1. **Rapidez de respuesta:** La constante de tiempo en lazo cerrado se reduce a $\tau_{lc} = \frac{\tau}{1 + K_p K}$. Como el denominador es mayor a 1, $\tau_{lc} < \tau$. Esto significa que a medida que se aumenta la ganancia K_p , el sistema responde y se estabiliza más rápido ante cambios en la referencia.
2. **Error de estado estacionario (e_{ss}):** La ganancia en estado estacionario del lazo cerrado es $K_{lc} = \frac{K_p K}{1 + K_p K}$. Para una entrada tipo escalón unitario, el valor final que alcanza el sistema es exactamente K_{lc} . Dado que el denominador siempre es mayor que el numerador ($1 + K_p K > K_p K$), la ganancia nunca alcanza el valor de 1. Esto genera un **error permanente** entre la referencia y la salida real, calculado como:

$$e_{ss} = 1 - K_{lc} = 1 - \frac{K_p K}{1 + K_p K} = \frac{1}{1 + K_p K} \quad (1.28)$$

Aunque aumentar K_p reduce este error, nunca llega a ser cero únicamente con acción proporcional.

1.3.2 Análisis del Control Proporcional-Integral (PI)

Como se evidenció en el análisis anterior, el controlador proporcional mejora la rapidez de respuesta del sistema, pero es incapaz de eliminar el error en estado estacionario ante una entrada escalón. Para solucionar este problema, se introduce una acción integral, dando lugar al controlador PI, cuya función de transferencia es $C(s) = K_p + \frac{K_i}{s}$. El esquema general se ilustra en la Figura 1.15.

Para eliminar este error, se incorpora una acción integral, dando lugar al controlador proporcional-integral (PI), cuya ley de control en el dominio del tiempo se define como:

$$u(t) = K_p e(t) + K_i \int_0^t e(\tau) d\tau \quad (1.29)$$

En el dominio de Laplace, el controlador PI se expresa como:

$$C(s) = K_p + \frac{K_i}{s} \quad (1.30)$$

La acción integral acumula el error a lo largo del tiempo, lo que permite corregir completamente el error en estado estacionario. Sin embargo, esta acumulación también introduce un polo en el origen, lo que aumenta el orden del sistema y puede afectar la estabilidad y la respuesta transitoria si no se sintoniza adecuadamente.

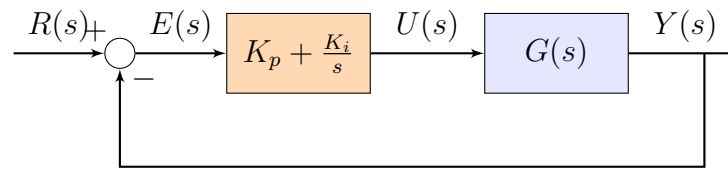


Figura 1.15: Diagrama de bloques de un sistema de control de lazo cerrado con controlador PI.

Para analizar el efecto de este controlador, se asume nuevamente la planta genérica de primer orden $G(s) = \frac{K}{\tau s + 1}$. Sustituyendo el controlador PI y la planta en la expresión general de lazo cerrado:

$$G_{lc}(s) = \frac{C(s)G(s)}{1 + C(s)G(s)} = \frac{\left(\frac{K_p s + K_i}{s}\right) \left(\frac{K}{\tau s + 1}\right)}{1 + \left(\frac{K_p s + K_i}{s}\right) \left(\frac{K}{\tau s + 1}\right)} \quad (1.31)$$

Desarrollando la expresión, se obtiene la función de transferencia en lazo cerrado:

$$G_{lc}(s) = \frac{(K_p s + K_i)K}{s(\tau s + 1) + (K_p s + K_i)K} = \frac{K K_p s + K K_i}{\tau s^2 + (1 + K K_p)s + K K_i} \quad (1.32)$$

De este modelo matemático se extraen las dos características fundamentales que definen al control PI:

1. **Cambio en la dinámica (de primer a segundo orden):** La presencia del polo en el origen introducido por la acción integral aumenta el orden del denominador. Esto implica que el sistema en lazo cerrado ahora posee una dinámica de segundo orden, lo que permite respuestas más ricas, pero introduce la posibilidad de oscilaciones y sobreimpulsos si las ganancias K_p y K_i no se sintonizan adecuadamente.
2. **Eliminación del error de estado estacionario (e_{ss}):** Para una entrada escalón ($R(s) = 1/s$), el error en estado estacionario puede evaluarse de manera analítica y general utilizando el teorema del valor final:

$$\begin{aligned}
 e_{ss} &= \lim_{t \rightarrow \infty} e(t) = \lim_{s \rightarrow 0} sE(s) = \lim_{s \rightarrow 0} s \cdot \frac{R(s)}{1 + C(s)G(s)} \\
 &= \lim_{s \rightarrow 0} s \cdot \frac{1/s}{1 + C(s)G(s)} = \lim_{s \rightarrow 0} \frac{1}{1 + C(s)G(s)} \\
 &= \lim_{s \rightarrow 0} \frac{1}{1 + \left(K_p + \frac{K_i}{s}\right) G(s)} \\
 &= \lim_{s \rightarrow 0} \frac{1}{1 + K_p G(s) + \frac{K_i}{s} G(s)} \\
 &\text{con } G(0) < \infty \\
 &= \lim_{s \rightarrow 0} \frac{1}{1 + K_p G(0) + \frac{K_i}{s} G(0)} \\
 &= \lim_{s \rightarrow 0} \frac{1}{1 + K_p G(0) + \infty} \\
 &= \frac{1}{\infty} \\
 &= 0
 \end{aligned} \tag{1.33}$$

Como se observa en el desarrollo matemático, al evaluar el límite cuando $s \rightarrow 0$ (que representa el sistema en estado estacionario), el término de la acción integral ($\frac{K_i}{s}$) tiende a infinito. Esto significa que el controlador proporciona teóricamente una ganancia infinita frente a señales constantes en el tiempo, como lo es la entrada escalón.

Al encontrarse esta ganancia infinita en el denominador de la función del error, fuerza matemáticamente a que el resultado sea nulo:

$$e_{ss} = 0 \tag{1.34}$$

En términos prácticos, esto demuestra que la acción integral suma de forma continua cualquier error por más pequeño que sea. El controlador seguirá modificando la señal de control $u(t)$ hasta que la salida del sistema alcance exactamente la referencia, eliminando por completo el error permanente que presentaba la acción únicamente proporcional.

1.3.3 Análisis del Control Proporcional-Integral-Derivativo (PID)

El controlador proporcional-integral-derivativo (PID) combina las tres acciones de control fundamentales, permitiendo mejorar simultáneamente la rapidez de respuesta, eliminar el error en estado estacionario y anticipar cambios en la dinámica del sistema.

Su ley de control en el dominio del tiempo se define como:

$$u(t) = K_p e(t) + K_i \int_0^t e(\tau) d\tau + K_d \frac{de(t)}{dt} \quad (1.35)$$

En el dominio de Laplace, el controlador PID se expresa como:

$$C(s) = K_p + \frac{K_i}{s} + K_d s \quad (1.36)$$

El esquema general del sistema en lazo cerrado se muestra en la Figura 1.16.

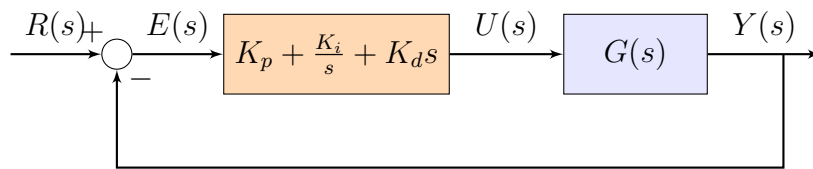


Figura 1.16: Diagrama de bloques con controlador PID.

Para analizar su efecto, se considera nuevamente una planta de primer orden:

$$G(s) = \frac{K}{\tau s + 1} \quad (1.37)$$

Sustituyendo en la expresión de lazo cerrado:

$$G_{lc}(s) = \frac{C(s)G(s)}{1 + C(s)G(s)} = \frac{(K_d s^2 + K_p s + K_i) K}{\tau s^2 + s + (K_d s^2 + K_p s + K_i) K} \quad (1.38)$$

El sistema resultante es de orden superior, lo que permite una mayor capacidad de ajuste dinámico.

Efecto de cada acción de control

El comportamiento del sistema puede ajustarse mediante las tres acciones del controlador PID:

- **Acción proporcional (K_p):** aumenta la rapidez de respuesta y reduce el error, aunque no lo elimina completamente.
- **Acción integral (K_i):** elimina el error en estado estacionario, pero puede introducir oscilaciones si es elevada.
- **Acción derivativa (K_d):** mejora el amortiguamiento del sistema, reduciendo el sobreimpulso y anticipando cambios en la respuesta.

Una sintonización adecuada de estos parámetros logra un compromiso entre rapidez, estabilidad y precisión.

Capítulo 2

Práctica 1: Control de velocidad con el Qube Servo 3

Modalidad: Presencial

Valoración: 10

Grupos: Máx. 2 o 3 estudiantes

Entrega: Según calendario

2.1 Introducción

El control de sistemas dinámicos es un componente fundamental en robótica, ya que permite regular variables físicas como posición, velocidad o fuerza en función de una referencia deseada. En aplicaciones reales, como manipuladores robóticos o sistemas autónomos, el desempeño del controlador influye directamente en la precisión, estabilidad y rapidez de los movimientos.

En esta práctica se aborda el problema del control de velocidad de un motor de corriente continua utilizando el sistema Qube-Servo 3. Como primer paso, se realiza la identificación experimental del modelo dinámico del sistema a partir de su respuesta al escalón en lazo abierto, permitiendo aproximarlos mediante un modelo de primer orden.

A partir de este modelo, se implementan diferentes estrategias de control en lazo cerrado, comenzando con un controlador proporcional (P), seguido de un controlador proporcional-integral (PI), con el objetivo de analizar su efecto sobre la dinámica del sistema, especialmente en términos de rapidez de respuesta y error en estado estacionario.

Finalmente, se introduce el controlador proporcional-integral-derivativo (PID), el cual será sintonizado de manera experimental como parte del reto de la práctica, permitiendo comprender el impacto de cada acción de control en el comportamiento del sistema.

2.2 Objetivos de aprendizaje

Objetivos de Aprendizaje

Objetivo general

Diseñar e implementar controladores de velocidad para el sistema Qube-Servo 3, a partir de la identificación experimental de su modelo dinámico.

Objetivos específicos

- OE1.** Obtener un modelo dinámico aproximado del sistema mediante la respuesta al escalón en lazo abierto.
- OE2.** Diseñar e implementar un controlador proporcional (P), proporcional-integral (PI) y evaluar su efecto sobre la rapidez de respuesta y el error en estado estacionario.
- OE3.** Implementar y ajustar experimentalmente un controlador PID y evaluar el efecto de cada parámetro sobre el desempeño del sistema.

2.3 Materiales requeridos

Materiales y Equipos

- QUBE-Servo 3 de Quanser
- Computadora con MATLAB/Simulink
- Add-on de MATLAB Coder y Simulink Coder
- Compilador de C/C++ compatible (p. ej., MinGW o Visual Studio 2022 Build Tools - recomendado)
- QUARC y controladores de hardware
- Cables de conexión y adaptadores

2.4 Procedimiento

i Nota

- Antes de comenzar, asegúrese de que el entorno de desarrollo esté correctamente configurado y que el QUBE-Servo 3 esté conectado y reconocido por el sistema.
- El material de referencia para esta práctica se encuentra disponible en el repositorio del curso [↗](#).

2.4.1 Control de velocidad a lazo abierto

A continuación se presenta el procedimiento para la implementación de la prueba de respuesta al escalón en lazo abierto, así como la identificación del modelo experimental a partir de los datos obtenidos para aproximar la función de transferencia a un sistema de primer orden. El diagrama de bloques general para esta etapa se muestra en la Figura 2.7.

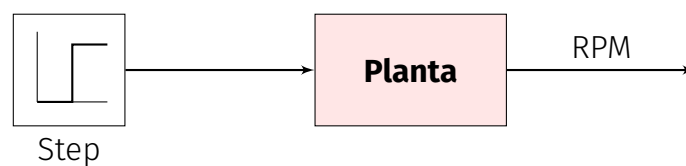


Figura 2.1: Diagrama de bloques simplificado en Lazo Abierto.

1. Completar los pasos de la sección 1.1.1 para establecer la conexión entre Simulink y el sistema Qube-Servo 3.
2. Agregar los bloques *HIL Read Encoder Timebase* y *Display*, conectarlos y ejecutar el experimento

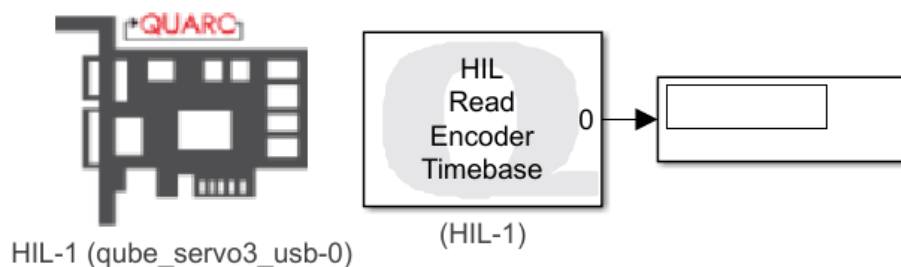


Figura 2.2: Bloques para lectura de encoder y visualización en Display.

3. **Posición:** Agregar dos bloques de *Gain* y configurar, dando doble clic, esas ganancias para realizar la conversión de las lecturas del encoder a:

- Rad: $K_{\text{rad}} = \frac{2\pi}{512 \times 4}$
- °: $K_{\text{°}} = \frac{360}{512 \times 4}$

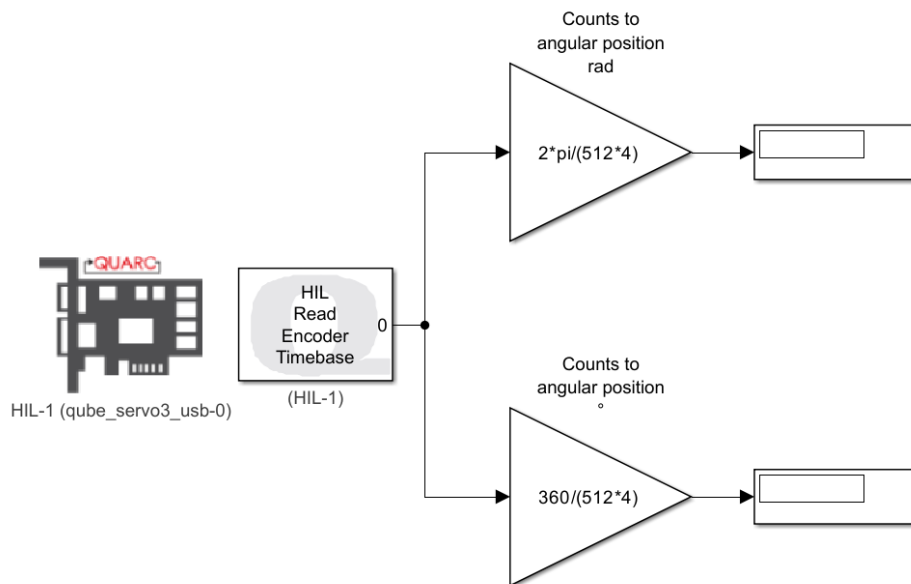


Figura 2.3: Bloques de ganancia para conversión de encoder a posición en radianes y grados.

4. Ejecutar el experimento y visualizar las lecturas de posición en los bloques *Display*.

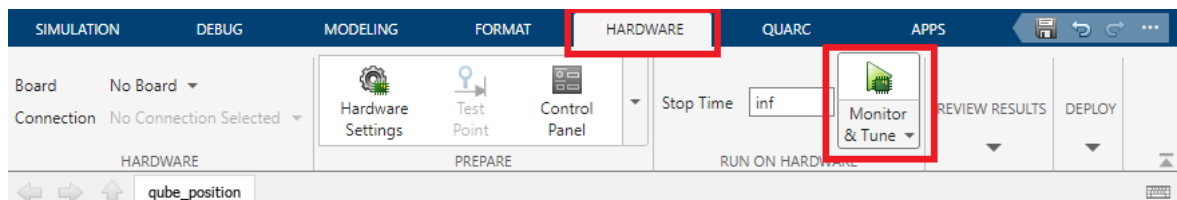


Figura 2.4: Ejemplo de ejecución del modelo en Simulink para el control de posición del QUBE-Servo 3.

5. **Velocidad:** Reemplazar el bloque de lectura del Qube *HIL Read Encoder Timebase* por el bloque *HIL Read Timebase*, configurándolo como se muestra en la Figura 2.5.

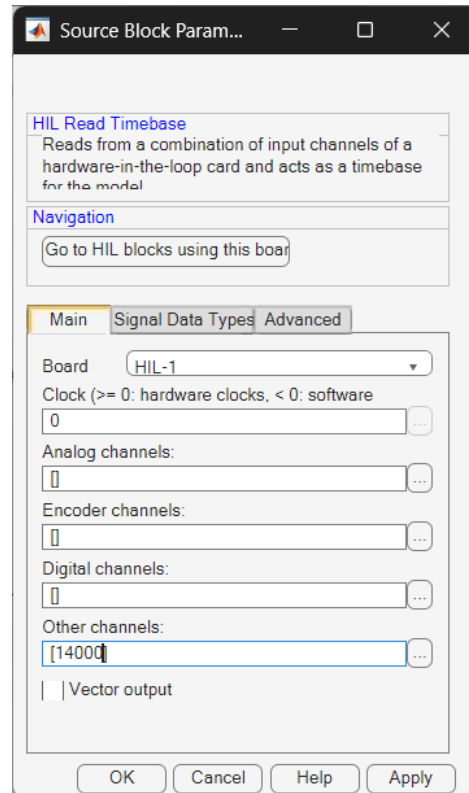


Figura 2.5: Configuración del bloque HIL Read Timebase para la lectura de la velocidad del sistema.

6. Agregar dos bloques *Gain* y configurarlos para realizar la conversión a RPM tanto de rad/s o °/s (ver Figura 2.6).

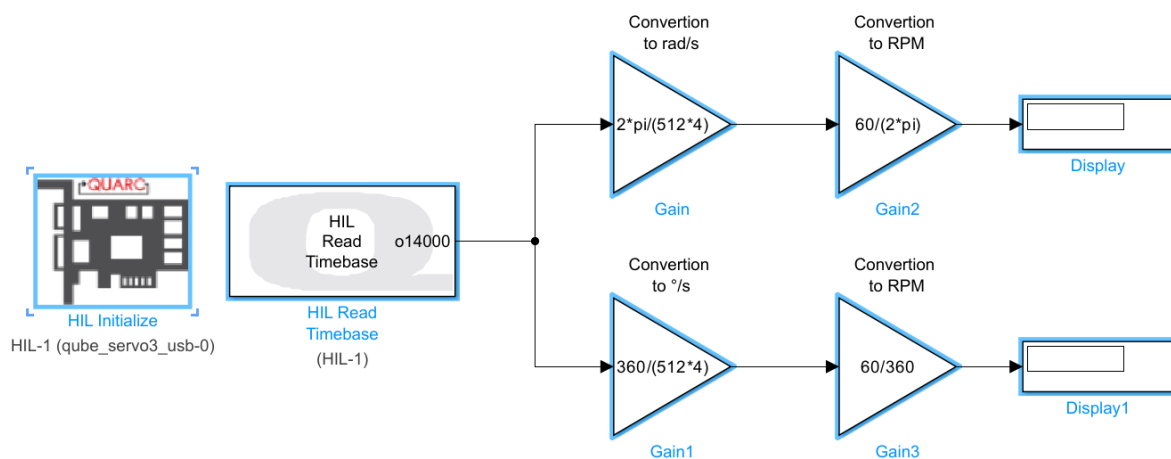


Figura 2.6: Bloques para cálculo de velocidad a partir de la derivada de la posición.

7. Reemplazar los *Display* por *Scopes*. Agregar un bloque *Step* y conectarlo al bloque *HIL Write Analog* para realizar una prueba de respuesta al escalón. Configurar el bloque *Step* para que realice un cambio de 0V a 8V a los 1s y ejecutar el experimento (ver Figura 2.7).

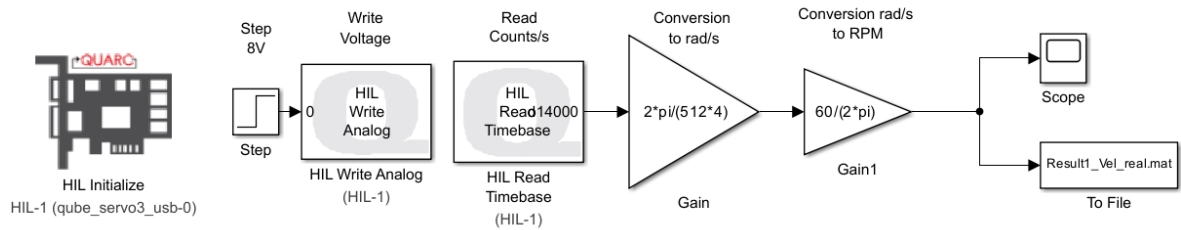


Figura 2.7: Diagrama de bloques para la prueba de respuesta al escalón en lazo abierto.

8. Ejecutar el script *Item2_Results_Real_Model.m* para aproximar el modelo a un sistema de primer orden:

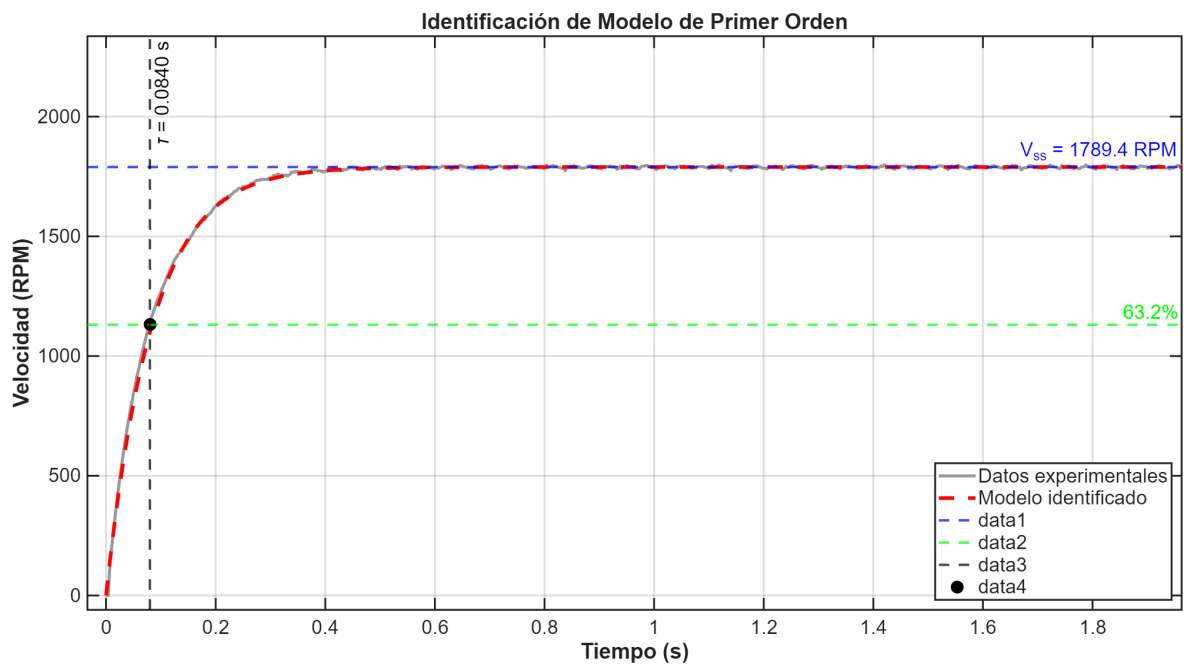


Figura 2.8: Resultados de la aproximación del modelo experimental a un sistema de primer orden.

El modelo obtenido es:

$$G_{\text{exp}}(s) \approx \frac{223.42}{0.0807s + 1} \quad (2.1)$$

2.4.2 Control de velocidad a lazo cerrado

Controlador proporcional

Al aplicar el controlador es necesario limitar la señal de control para evitar saturación del actuador y proteger el hardware. Para esto, se utiliza un bloque de *Saturation* configurado con los límites operativos de voltaje del sistema (−15 V a 15 V). El esquema del controlador proporcional se muestra en la Figura 2.9.

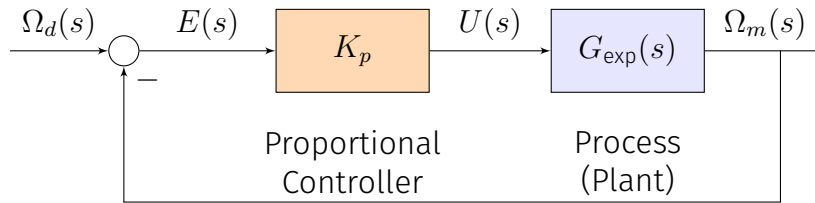


Figura 2.9: Diagrama de bloques de un sistema en lazo cerrado con controlador proporcional.

Sustituyendo el modelo experimental identificado previamente:

$$G_{\text{exp}}(s) = \frac{223.42}{0.0807s + 1} \quad (2.2)$$

en la expresión general de lazo cerrado, se obtiene:

$$\frac{\Omega_m(s)}{\Omega_d(s)} = \frac{K_p \left(\frac{223.42}{0.0807s + 1} \right)}{1 + K_p \left(\frac{223.42}{0.0807s + 1} \right)} = \frac{223.42K_p}{0.0807s + 1 + 223.42K_p} \quad (2.3)$$

Agrupando términos para llevarla a su forma estándar de primer orden:

$$\frac{\Omega_m(s)}{\Omega_d(s)} = \frac{\left(\frac{223.42K_p}{1 + 223.42K_p} \right)}{\left(\frac{0.0807}{1 + 223.42K_p} \right)s + 1} = \frac{K_{lc}}{\tau_{lc}s + 1} \quad (2.4)$$

Para el experimento, se propone un criterio de diseño basado en la asignación de la constante de tiempo. Se requiere que el sistema en lazo cerrado responda **al doble de velocidad** (reduciendo la constante de tiempo a la mitad) en comparación con el sistema en lazo abierto. Por lo tanto, la constante de tiempo deseada es:

$$\tau_{lc} = \frac{0.0807}{2} = 0.04035 \text{ s} \quad (2.5)$$

Igualando esta condición con el denominador de la función analítica:

$$\frac{0.0807}{1 + 223.42K_p} = 0.04035 \implies 1 + 223.42K_p = 2 \quad (2.6)$$

Despejando la ganancia proporcional requerida:

$$K_p = \frac{1}{223.42} \approx 0.00448 \quad (2.7)$$

Como consecuencia directa de esta sintonización, evaluando la ganancia en estado estacionario K_{lc} de la ecuación estándar, se espera un valor final de:

$$\Omega_m(\infty) = \Omega_d \left(\frac{223.42(0.00448)}{1 + 223.42(0.00448)} \right) = \Omega_d \left(\frac{1}{1 + 1} \right) = 0.5 \Omega_d \quad (2.8)$$

Esto demuestra que, para estabilizar el sistema al doble de velocidad, la salida se quedará exactamente en la mitad del valor de referencia R (un error estacionario del 50%).

A continuación se presenta el procedimiento para la implementación del controlador proporcional en Simulink.

1. Sobre el programa de lazo abierto, eliminar el *Step*.
2. Agregar un bloque *Constant* con un valor de 1789, que fueron los resultados obtenidos del experimento en lazo abierto con 8V.
3. Agregar un bloque *Sum* y configurar una resta.
4. Agregar un bloque *Saturation*, configurarlo con 15 y -15.
5. Modificar el nombre del bloque *To File* para cambiar el archivo de salida a *close_loop_Kp.mat*
6. Ordenar los bloques de acuerdo a la Figura 2.10 y ejecutar el experimento para observar la respuesta del sistema en lazo cerrado con el controlador proporcional implementado.

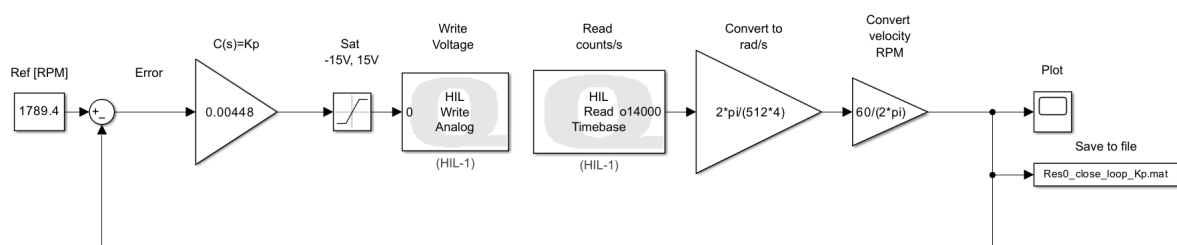


Figura 2.10: Diagrama de bloques para la implementación del controlador proporcional en lazo cerrado.

7. Finalmente, ejecutar el script *Results_close_loop_Kp.m* para analizar los resultados obtenidos y contrastar la nueva constante de tiempo y el error de estado estacionario con los cálculos teóricos.

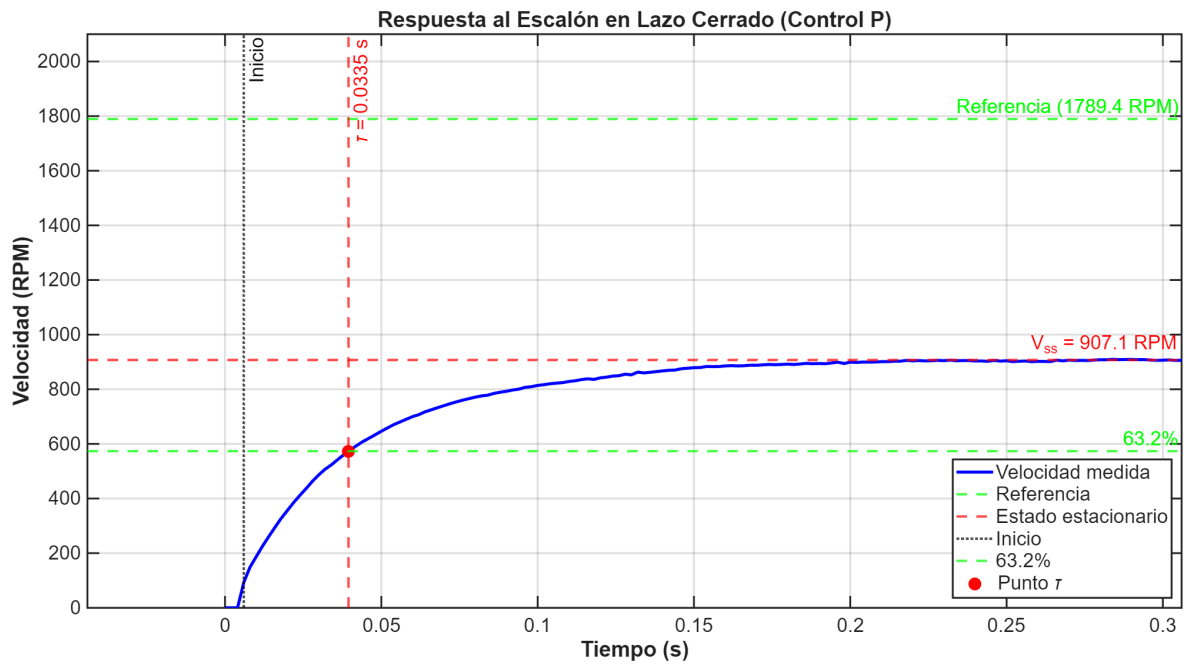


Figura 2.11: Resultados de la respuesta del sistema en lazo cerrado con controlador proporcional.

Como resultado del experimento se disminuyó a poco más de la mitad el tiempo de respuesta del sistema, pero se evidenció un error en estado estacionario de aproximadamente 50% respecto a la referencia, tal como se predijo teóricamente.

Implementación de Control Proporcional-Integral (PI)

Para la corrección del error estacionario observado en la etapa anterior, se implementa una acción integral, dando lugar al controlador PI. El esquema se presenta en la Figura 2.12.

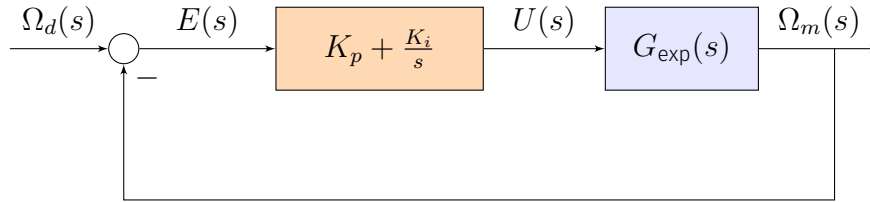


Figura 2.12: Diagrama de bloques del sistema con controlador PI.

Considerando el modelo experimental $G_{\text{exp}}(s) = \frac{223.42}{0.0807s+1}$, la función de transferencia de lazo cerrado a implementar se calcula directamente como:

$$G_{lc}(s) = \frac{C(s)G_{\text{exp}}(s)}{1 + C(s)G_{\text{exp}}(s)} = \frac{223.42(K_p s + K_i)}{0.0807s^2 + (1 + 223.42K_p)s + 223.42K_i} \quad (2.9)$$

Esta ecuación demuestra que el sistema pasó de tener una dinámica de primer orden a una de segundo orden, lo que permitirá eliminar el error en estado estacionario pero introducirá un comportamiento transitorio diferente (posible sobreimpulso).

A continuación se presenta el procedimiento para la implementación del controlador PI en Simulink.

1. Agregar un *Transfer Function* con la función de transferencia del controlador I, con una ganancia $K_i = 0.0555$, y colocarlo en paralelo para sumarlo con el controlador proporcional.

El valor de K_i se obtiene a partir de la relación:

$$K_i = \frac{K_p}{T_i} \quad (2.10)$$

donde T_i es la constante de tiempo de la acción integral. Para este caso, se toma:

$$T_i \approx \tau \quad (2.11)$$

por lo que:

$$K_i = \frac{K_p}{\tau} = \frac{0.00448}{0.0807} \approx 0.0555 \quad (2.12)$$

2. Agregar un bloque *Sum of elements* para sumar la salida del bloque proporcional y el bloque integral, configurándolo para realizar una suma.

3. Agregar el bloque *Saturation* para limitar la salida del controlador.
4. Conectar los bloques de acuerdo a la Figura 2.13 y ejecutar el experimento para observar la respuesta del sistema con el controlador PI implementado.

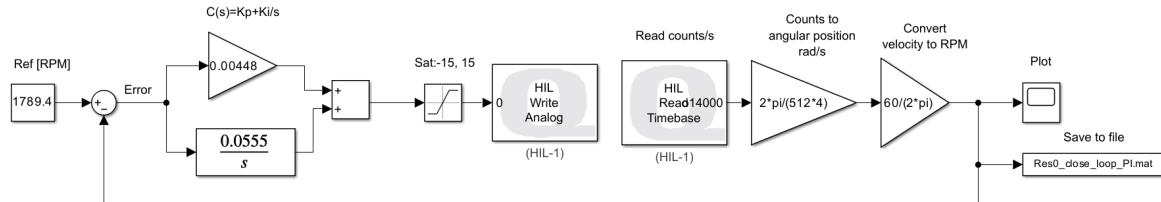


Figura 2.13: Diagrama de bloques para la implementación del controlador PI en Simulink.

5. Ejecutar el script *Item1_Results_PI_Controller.m* para analizar los resultados obtenidos.

El resultado de la Figura 2.14 muestra que el sistema logró eliminar el error en estado estacionario, alcanzando la referencia de 1976 RPM. Sin embargo, se observa un sobreimpulso del 20% y un tiempo de establecimiento de aproximadamente 0.1 segundos, lo que evidencia el impacto de la acción integral en la dinámica del sistema.

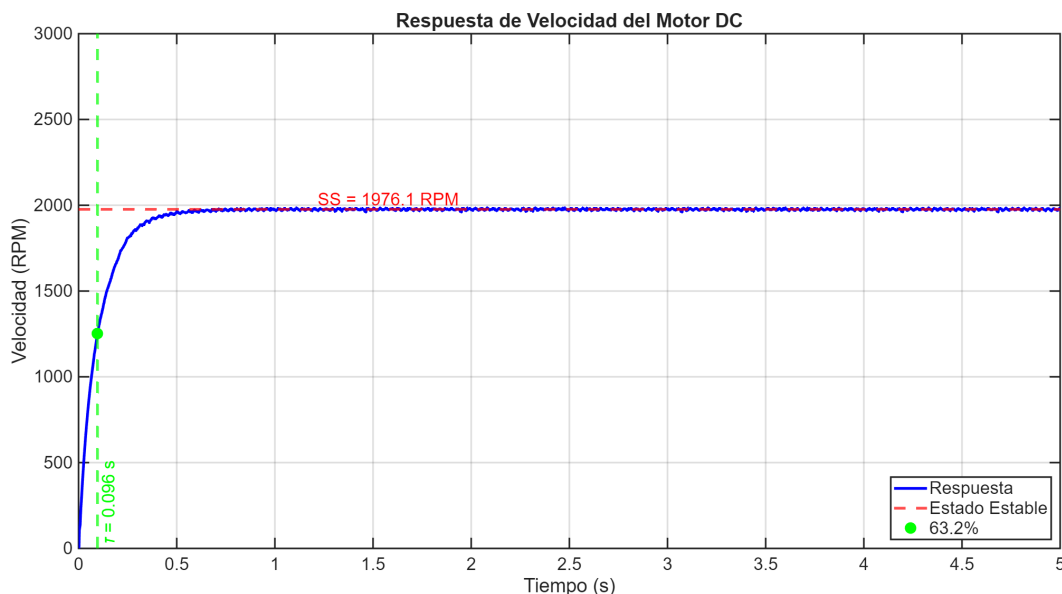


Figura 2.14: Resultados de la respuesta del sistema en lazo cerrado con controlador PI.

Al incrementar K_i , se observa un aumento en el sobreimpulso y una reducción en el tiempo de establecimiento, pero con una respuesta más oscilatoria. Por otro lado, al disminuir K_i , se reduce el sobreimpulso y se obtiene una respuesta más suave, pero con un tiempo de establecimiento más largo. Esto demuestra

la importancia de la sintonización adecuada del controlador PI para lograr un equilibrio entre rapidez de respuesta y estabilidad.

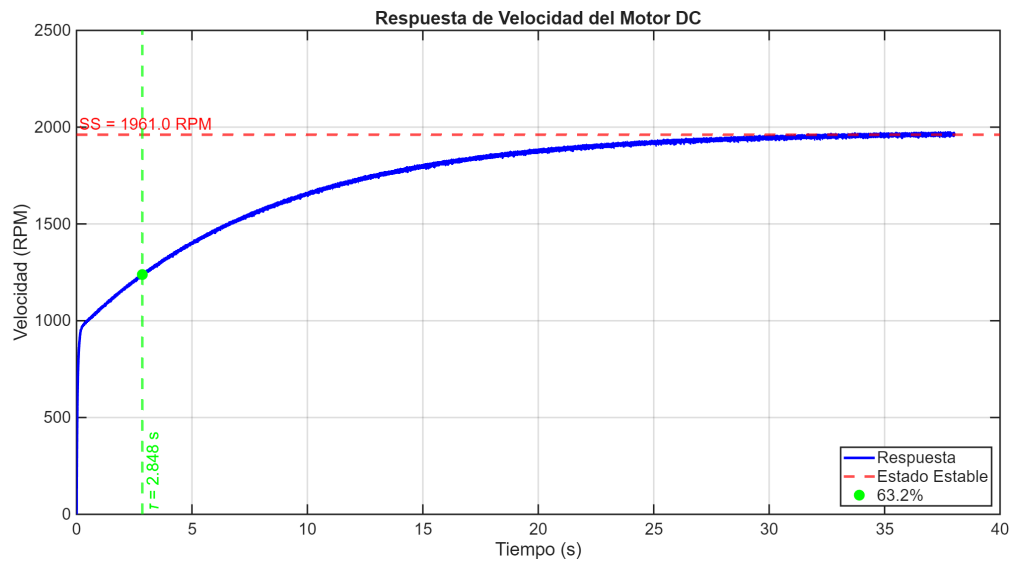


Figura 2.15: Respuesta del sistema con ganancia $K_i=0.001$

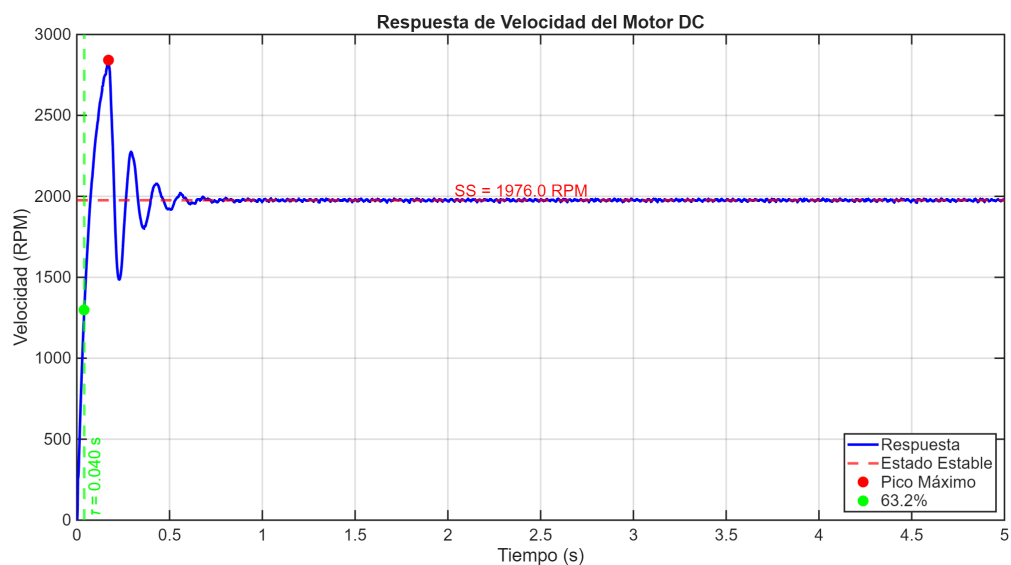


Figura 2.16: Respuesta del sistema con ganancia $K_i=1$

Sintonización

Existen diversos métodos clásicos para la sintonización de controladores PI, tales como el método de Ziegler-Nichols, Cohen-Coon, entre otros, los cuales permiten obtener valores iniciales de las ganancias a partir de la respuesta del sistema. En este caso analizaremos de forma experimental la variación de la respuesta del sistema al modificar los parámetros K_p y K_i .

2.5 Reto

En esta sección se propone como reto la implementación y sintonización experimental de un controlador Proporcional-Integral-Derivativo (PID) para velocidad aplicado al modelo identificado de la planta.

Modelo del controlador PID

La ley de control en el dominio del tiempo está dada por:

$$u(t) = K_p e(t) + K_i \int_0^t e(\tau) d\tau + K_d \frac{de(t)}{dt} \quad (2.13)$$

El controlador PID en el dominio de Laplace se implementa en un lazo cerrado con saturación en la señal de control:

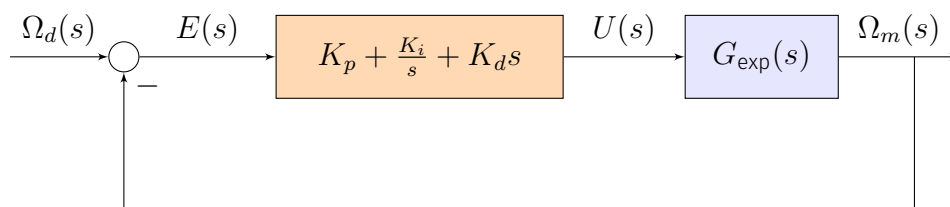


Figura 2.17: Sistema en lazo cerrado con controlador PID.

Objetivo del reto

Implementar un controlador PID para el sistema de control de velocidad del Qube Servo 3, partiendo del modelo identificado previamente y ajustando experimentalmente las ganancias K_p , K_i y K_d para mejorar el desempeño del sistema.

Procedimiento

1. Partir del controlador PI previamente diseñado.
2. Introducir la acción derivativa K_d con un valor pequeño.
3. Ajustar iterativamente K_p , K_i y K_d .

Criterios de análisis

Durante la sintonización, evaluar:

- Sobreimpulso
- Tiempo de establecimiento
- Rapidez de respuesta
- Error en estado estacionario

2.6 Preguntas de análisis

1. Compare la respuesta del sistema en lazo abierto y con control proporcional. ¿Cómo influye K_p en la constante de tiempo y en el error en estado estacionario?
2. Analice el efecto de la acción integral en el controlador PI. ¿Por qué el error en estado estacionario tiende a cero? ¿Qué impacto tiene el aumento de K_i sobre el sobreimpulso y la estabilidad del sistema?
3. En el controlador PID, describa el efecto individual de cada término (K_p , K_i , K_d) sobre la respuesta del sistema. Con base en sus pruebas experimentales, explique qué ocurre cuando:
 - K_i es muy alto
 - K_d es muy alto
 - K_d es muy bajo o nulo

2.7 Entregables

Entregables

1. **Demostración experimental:** Validación ante el docente del funcionamiento del sistema de control PID implementado en el Qube Servo 3.
2. **Informe técnico en formato IEEE**, que incluya:
 - **Implementación del reto:**
 - Implementación del controlador PID
 - Proceso de sintonización experimental de K_p , K_i y K_d
 - **Análisis de resultados:**
 - Gráficas de respuesta (lazo abierto, P, PI y PID)
 - Discusión y comparación del efecto de cada parámetro en: Tiempo de establecimiento, sobreimpulso, error en estado estacionario.
 - **Respuestas a las preguntas de análisis**
3. **Anexo de código:**
 - Modelos de Simulink desarrollados
 - Código debidamente comentado y organizado

2.8 Rúbrica de Evaluación

Cuadro 2.1: Rúbrica de evaluación — Práctica: Control de velocidad con Qube-Servo 3 (10 puntos)

Criterio	Peso	Nivel alto (9–10)	Nivel medio (6–8) / bajo (0–5)
Identificación del modelo	25 %	Obtiene correctamente el modelo de primer orden a partir de datos experimentales. Parámetros K y τ coherentes y validados con gráfica.	Modelo incompleto o con errores. Parámetros inconsistentes o sin validación clara.
Reto: Controlador PID	45 %	Implementa y sintoniza el PID de forma experimental. Evidencia clara del efecto de K_p , K_i y K_d en la respuesta del sistema.	PID incompleto o mal ajustado. No se evidencia análisis del efecto de las ganancias.
Informe y análisis	20 %	Informe claro, organizado y técnico (formato IEEE). Incluye gráficas, comparaciones y conclusiones coherentes.	Informe desordenado, incompleto o sin análisis suficiente. Falta de claridad en resultados.
Código y simulación	10 %	Modelos de Simulink y scripts correctamente estructurados, comentados y funcionales.	Código desordenado, sin comentarios o con errores de ejecución.

Capítulo 3

Práctica 2: Control de posición con el Qube Servo 3

Modalidad: Presencial

Valoración: 10

Grupos: Máx. 2 o 3 estudiantes

Entrega: Según calendario

3.1 Introducción

El control de posición en robótica es el proceso de regulación de la variable de salida (ángulo del eje) para que coincida con una referencia deseada mediante un lazo de retroalimentación cerrada. A nivel técnico, esto implica que el controlador procesa constantemente la señal de error $e(t)$, definida como la diferencia entre el setpoint y la posición real medida por sensores de alta precisión.

En nuestra estación de trabajo, el QUBE-Servo 3 de Quanser representa un sistema electromecánico de precisión de segundo orden. Este dispositivo integra un motor DC de imán permanente y un codificador óptico de cuadratura que proporciona una resolución de hasta 2048 pulsos por revolución.

El flujo de trabajo técnico se resume en:

- **Actuación:** El motor responde a una señal analógica de $\pm 10V$.
- **Retroalimentación:** El encoder traduce el desplazamiento angular en pulsos digitales para el procesador.
- **Compensación:** Aplicamos leyes de control para manipular la función de transferencia del sistema $G(s)$ y optimizar parámetros críticos como el tiempo de asentamiento (t_s) y el sobrepaso máximo (M_p).

3.2 Objetivos de aprendizaje

Objetivos de Aprendizaje

Objetivo general

Diseñar e implementar control de posición en el sistema QUBE-Servo 3 utilizando controladores P, PD y PID, y analizar el impacto de cada estrategia sobre el desempeño del sistema.

Objetivos específicos

- OE1.** Obtener y validar experimentalmente la relación entre las variables del sistema (posición y velocidad angular) a partir de pruebas en lazo abierto.
- OE2.** Implementar un sistema de control de posición en lazo cerrado utilizando un controlador proporcional (P) y proporcional-derivativo (PD), y analizar su efecto sobre el error en estado estacionario y la respuesta transitoria.
- OE3.** Implementar y ajustar experimentalmente un controlador Proporcional-Integral-Derivativo (PID) y evaluar el efecto de cada parámetro sobre el desempeño del sistema.

3.3 Materiales requeridos

Materiales y Equipos

- QUBE-Servo 3 de Quanser
- Computadora con MATLAB/Simulink
- Add-on de MATLAB Coder y Simulink Coder
- Compilador de C/C++ compatible (p. ej., MinGW o Visual Studio 2022 Build Tools - recomendado)
- QUARC y controladores de hardware
- Cables de conexión y adaptadores

3.4 Procedimiento

i Nota

- Antes de comenzar, asegúrese de que el entorno de desarrollo esté correctamente configurado y que el QUBE-Servo 3 esté conectado y reconocido por el sistema.
- El material de referencia para esta práctica se encuentra disponible en el [repositorio del curso](#) .

3.4.1 Control de posición a lazo abierto

Configuración mínima en Simulink

Para establecer una comunicación básica entre MATLAB/Simulink y el QUBE-Servo 3, se requiere de 3 bloques fundamentales: *HIL-Initialize*, *HIL Read Timebase* y *HIL Write*. A continuación, se describen los pasos para configurar estos bloques:

1. Completar los pasos de la sección 1.1.1 para establecer la configuración mínima entre Simulink y el sistema Qube-Servo 3.
2. Utilizar la salida 0 del *Encoder channels* del bloque *HIL Read Timebase* para leer la posición angular del disco, que se encuentra directamente disponible a través de un encoder integrado en el sistema.
3. Utilizar la salida 14000 de *Other channels* del bloque *HIL Read Timebase* para leer la velocidad angular del disco, que se encuentra directamente disponible a través de un tacómetro integrado en el sistema.

Esta configuración mínima permitirá establecer una comunicación básica entre el modelo de Simulink y el hardware del QUBE-Servo 3, lo que es esencial para implementar cualquier estrategia de control posteriormente. Luego, dependiendo la variable que se desee controlar (posición o velocidad angular), se debe convertir las señales de pulsos a las unidades correspondientes (radianes o radianes por segundo) utilizando un bloque de ganancia en Simulink. En la Figura 3.1 se muestra un ejemplo del modelo con la configuración mínima y la conversión de pulsos a radianes para la posición angular.

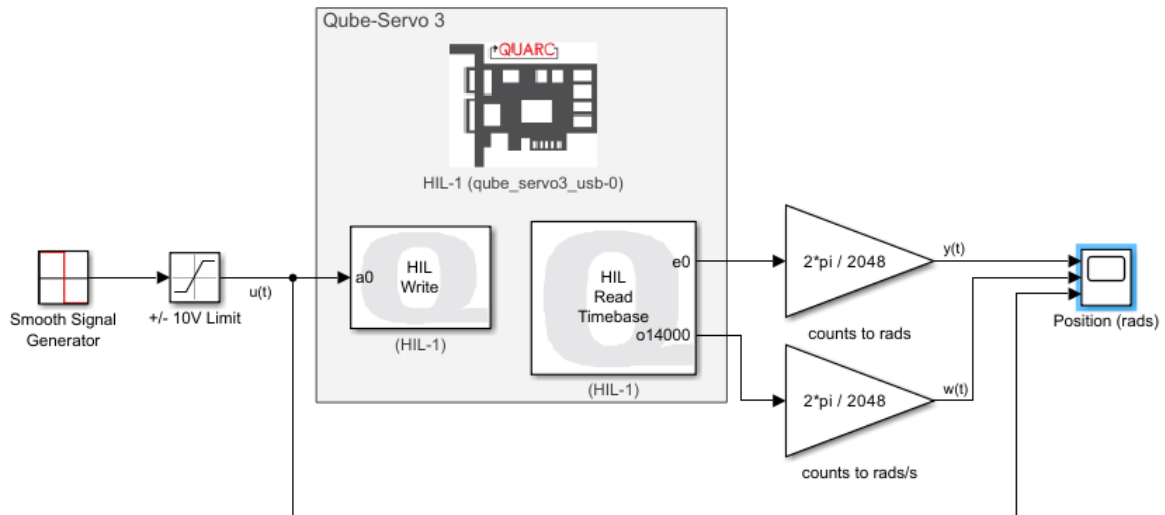


Figura 3.1: Ejemplo de configuración final del modelo en Simulink para el control de posición del QUBE-Servo 3.

Posición angular $\theta_m(t)$

Para la lectura de la posición angular, es importante tener en cuenta que la salida del encoder o del bloque *HIL Read Timebase* devuelve la señal en pulsos, por lo que se requiere una conversión a la unidad de medida deseada, en este caso radianes. La relación de conversión se basa en la cantidad total de pulsos por revolución del encoder, que para el QUBE-Servo 3 es de 2048 pulsos por revolución.

1. Agregar un bloque de función matemática para realizar la conversión de pulsos a radianes utilizando la siguiente fórmula:

$$\theta = \frac{2\pi \cdot P}{N} \quad (3.1)$$

Donde θ es el ángulo en radianes, P es el número de pulsos leídos por el encoder y N es la cantidad total de pulsos por revolución.

2. Conectar la salida del bloque *HIL Read Timebase* al bloque de función matemática para obtener la posición angular en radianes.
3. Asegurarse de que la señal de control enviada al bloque *HIL Write* esté correctamente escalada para que corresponda a la señal de voltaje requerida por el motor, es decir, en el rango de $\pm 10V$, en este caso se puede utilizar una función de saturación para limitar la señal.
4. Agregar un bloque de señal para generar un valor de entrada al proceso, por ejemplo, una constante, un bloque de escalón *Step* o una onda cuadrada generada con *Smooth Signal Generator*, etc.
5. Por último, para ejecutar el modelo, seleccione la pestaña *Hardware* o *QUARC* y presione *Monitor & Tune* , como se muestra en la Figura 3.2.

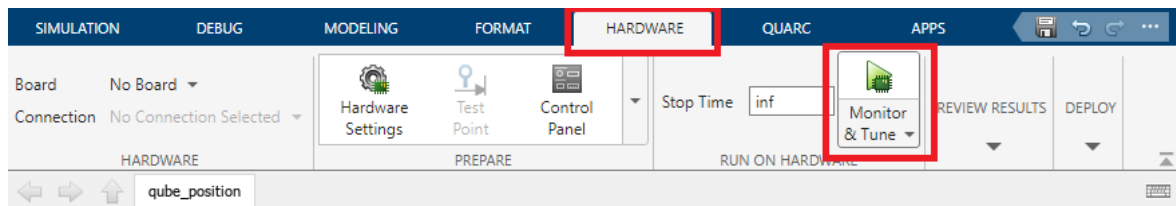


Figura 3.2: Ejemplo de ejecución del modelo en Simulink para el control de posición del QUBE-Servo 3.

En la Figura 3.1 se muestra un ejemplo de cómo podría quedar el modelo de Simulink con la configuración mínima y la conversión de pulsos a radianes, con una entrada de pulsos cuadrados directamente a la señal de control, la salida del sistema puede observarse en la Figura 3.3.

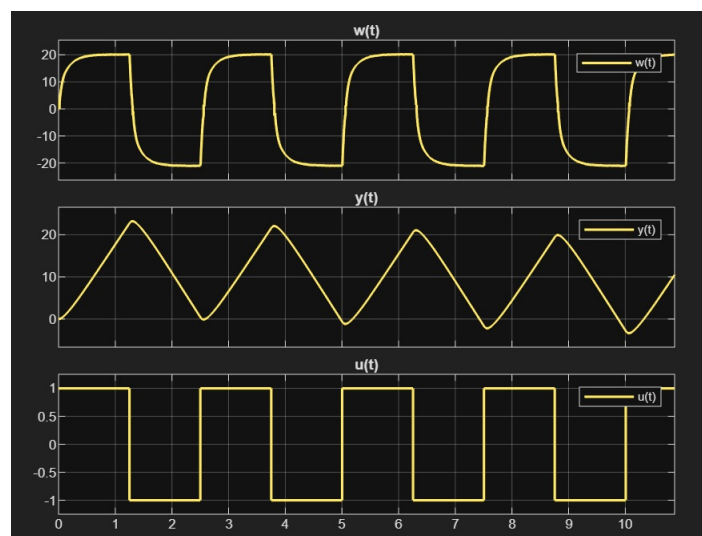


Figura 3.3: Ejemplo de salida del sistema en Simulink a lazo abierto para el control de posición del QUBE-Servo 3.

Velocidad angular $\omega_m(t)$

Para la lectura de la velocidad angular, se puede utilizar el mismo bloque *HIL Read Timebase* para leer el canal 014000. Al igual que con la posición angular, es necesario realizar una conversión de pulsos por segundo a radianes por segundo utilizando la siguiente fórmula:

$$\omega_m(t) = \frac{2\pi \cdot P}{N} \quad (3.2)$$

donde N es el número de pulsos por revolución y P es el número de pulsos por segundo.

Funciones de transferencia del sistema

Como resultado de las pruebas realizadas a lazo abierto y con base en la documentación oficial del QUBE-Servo 3, se puede obtener dos funciones de transferencia aproximadas del sistema, una para la posición angular y otra para la velocidad angular.

La función de transferencia de voltaje $V_m(s)$ a velocidad angular $\Omega_m(s)$, se puede expresar como:

$$\frac{\Omega_m(s)}{V_m(s)} = \frac{K}{\tau s + 1} \quad (3.3)$$

donde K es la ganancia estática del sistema y τ es la constante de tiempo. La respuesta del sistema real se puede observar en la Figura 3.4, correspondiente a $\omega_m(t)$, donde se puede apreciar que el sistema tiene un comportamiento de primer orden con una ganancia estática y una constante de tiempo que se pueden estimar a partir de la respuesta a lazo abierto. Para el QUBE-Servo 3, se ha reportado que $K \approx 24$ y $\tau \approx 0.1s$, aunque estos valores pueden variar dependiendo de las condiciones específicas del sistema y la configuración utilizada.

La función de transferencia de voltaje $V_m(s)$ a posición angular $\Theta_m(s)$, se puede obtener integrando la función de transferencia de velocidad, lo que da lugar a:

$$\frac{\Theta_m(s)}{V_m(s)} = \frac{K}{s(\tau s + 1)} \quad (3.4)$$

Esta función de transferencia representa un sistema de segundo orden con un polo en el origen (debido a la integración) y un polo en $s = -1/\tau$. La respuesta del sistema real a la posición angular también se puede observar en la Figura 3.4, correspondiente a $\theta_m(t)$, donde se puede apreciar que el sistema tiene un comportamiento de segundo orden con una ganancia estática y una constante de tiempo que se pueden estimar a partir de la respuesta a lazo abierto.

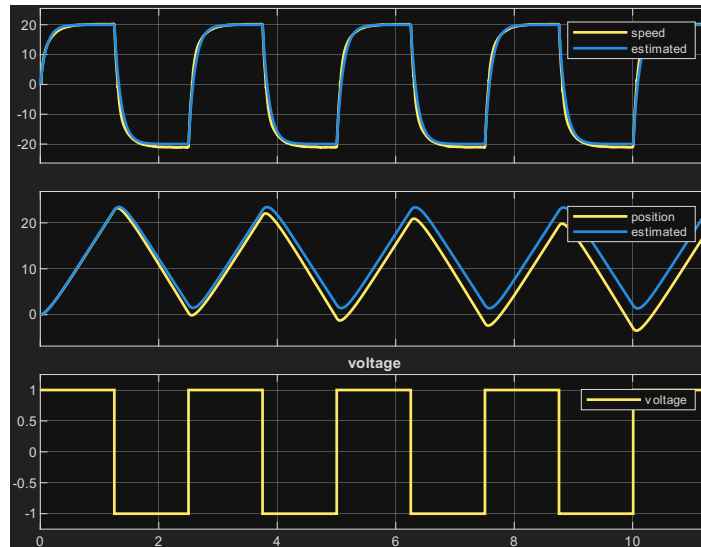


Figura 3.4: Ejemplo de salida del sistema en Simulink a lazo abierto para la función de transferencia de posición angular $\Theta_m(s)/V_m(s)$ y velocidad angular $\Omega_m(s)/V_m(s)$ del QUBE-Servo 3.

3.4.2 Control de posición a lazo cerrado

Controlador proporcional

El control proporcional (P) ajusta la señal de control en función del error instantáneo entre la referencia y la posición medida del sistema. Su ley de control se expresa como:

$$u(t) = K_p \cdot e(t) \quad (3.5)$$

donde K_p es la ganancia proporcional y $e(t)$ es el error de seguimiento, definido como $e(t) = \theta_d(t) - \theta_m(t)$. En este montaje, la señal aplicada al motor es $v_m(t) = u(t)$. La Figura 3.5 muestra el diagrama de bloques del lazo cerrado, en el que $G(s)$ representa la dinámica del sistema desde $v_m(t)$ hasta la salida angular $\theta_m(t)$.

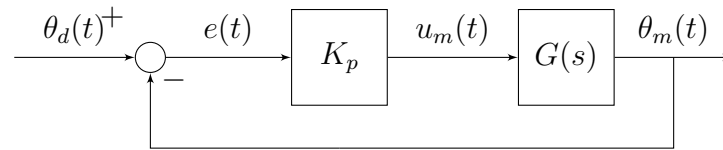


Figura 3.5: Diagrama de bloques del sistema de control a lazo cerrado con controlador proporcional K_p .

Con esta base, se implementa el controlador P en Simulink como primer escalón antes de pasar a estrategias más avanzadas (por ejemplo, PD o PID). El esquema de implementación se presenta en la Figura 3.6 y se resume en los siguientes pasos:

1. Agregar la retroalimentación de la posición angular medida al modelo, conectando la salida $y(t)$ a un bloque de resta para calcular la señal de error $e(t)$, que es la diferencia entre el setpoint (el bloque *Smooth Signal Generator*) y la posición angular medida.
2. Agregar un bloque de ganancia proporcional (P) al modelo, con un *Slider gain*. Este bloque tomará la señal de error $e(t)$ y la multiplicará por una ganancia proporcional K_p para generar la señal de control.
3. Conectar la salida del bloque de función de control proporcional (ganancia K_p) al bloque de saturación y luego *HIL Write*.

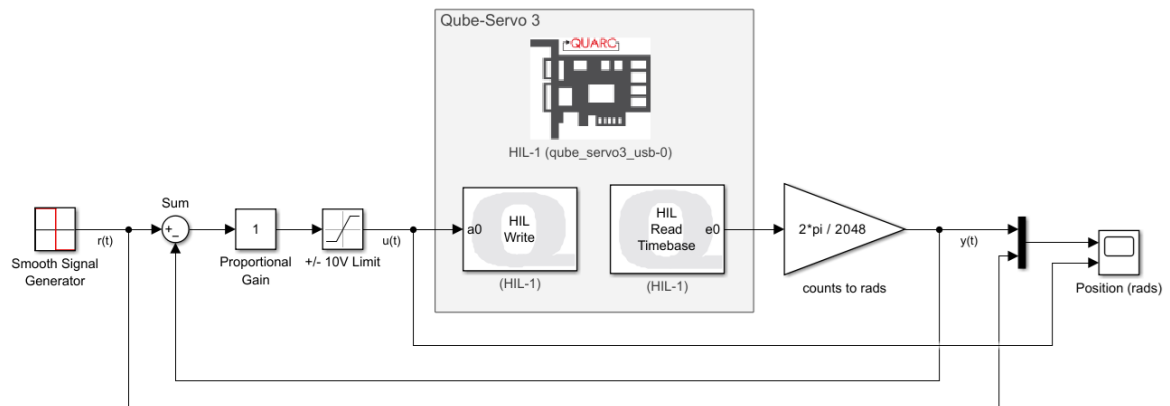


Figura 3.6: Ejemplo de configuración del modelo en Simulink para un controlador proporcional (P) en el control de posición del QUBE-Servo 3.

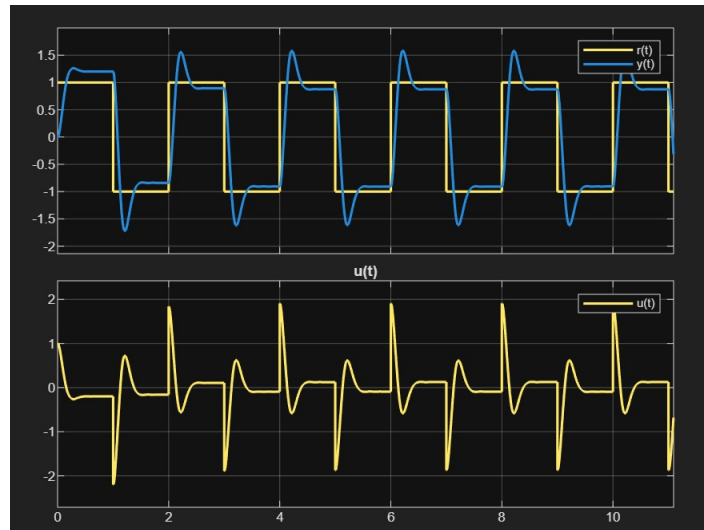


Figura 3.7: Respuesta del sistema con un controlador proporcional (P) para $K_p = 1$.

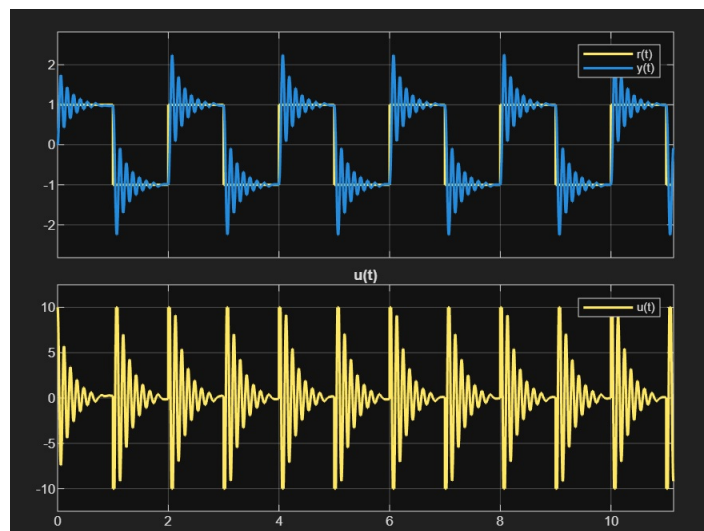


Figura 3.8: Respuesta del sistema con un controlador proporcional (P) para $K_p = 10$.

Ejercicio

Ajustar la ganancia proporcional K_p para observar cómo afecta a la respuesta del sistema. Se recomienda comenzar con un valor pequeño y aumentarlo gradualmente hasta obtener una respuesta estable y rápida sin oscilaciones excesivas.

En las figuras 3.7 y 3.8 se muestra la respuesta del sistema a una entrada de referencia de señal cuadrada de 0.4Hz y $\pi/4$ de amplitud, para un valor de ganancia proporcional $K_p = 1$ y $K_p = 10$, respectivamente. Se puede observar que para esta ganancia el sistema alcanza el setpoint con un tiempo de asentamiento razonable y sin oscilaciones excesivas, pero mantiene un error en estado estacionario, mientras que para valores más altos, la respuesta es más rápida y con un error menor, pero presenta oscilaciones.

Controlador Proporcional-Derivativo

El control proporcional-derivativo (PD) añade una acción derivativa al control proporcional para mejorar el compromiso entre rapidez y amortiguamiento. Su forma ideal se escribe como:

$$u(t) = K_p \cdot e(t) + K_d \cdot \frac{de(t)}{dt} \quad (3.6)$$

donde K_p es la ganancia proporcional, K_d es la ganancia derivativa y $e(t)$ es el error de seguimiento. En la implementación práctica no se deriva directamente $e(t)$, ya que cambios bruscos en la referencia pueden producir «patada derivativa». Por ello, se utiliza la velocidad angular medida $\omega_m(t)$ (derivada de la salida) en la rama derivativa. La estructura empleada se muestra en la Figura 3.9, en la que $G(s)$ modela la dinámica desde $v_m(t)$ hasta $\omega_m(t)$ y el bloque integrador entrega $\theta_m(t)$ para la realimentación de posición.

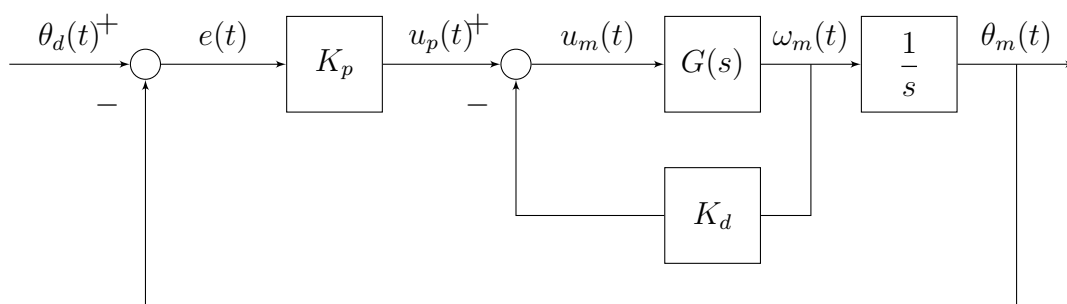


Figura 3.9: Diagrama de bloques del control a lazo cerrado con controlador proporcional-derivativo (PD).

Respecto al controlador P, el término derivativo permite reducir oscilaciones y mejorar el comportamiento transitorio cuando se ajusta adecuadamente. En Simulink, la implementación práctica se realiza con una rama proporcional y una rama derivativa basada en la velocidad angular medida por el tacómetro del bloque *HIL-Timebase Read* (canal 14000 en *Other channels*), tal como se ilustra en la Figura 3.10. Los pasos sugeridos son:

1. Agregar la señal de velocidad angular $\omega(t)$ del QUBE-Servo 3 al modelo, utilizando el bloque *HIL-Timebase Read* para leer el canal del tacómetro (canal 14000), que viene expresado en pulsos por segundo.
2. Convertir la señal de velocidad angular de pulsos por segundo a radianes por segundo utilizando el mismo bloque de función matemática utilizado para la conversión de posición angular.
3. Agregar un bloque de ganancia para el término derivativo (D) al modelo, con un *Slider gain*. Este bloque tomará la señal de velocidad angular $\omega(t)$ y la multiplicará por una ganancia derivativa K_d para generar la señal de control derivativa.
4. Agregar la retroalimentación del término derivativo al bloque de control proporcional, restando la señal de control derivativa a la señal de control proporcional para obtener la señal de control total que se enviará al bloque de saturación y luego a *HIL Write*.
5. Se recomienda que los bloques de ganancia tengan un límite de 10 y 1 para la ganancia proporcional y derivativa, respectivamente.

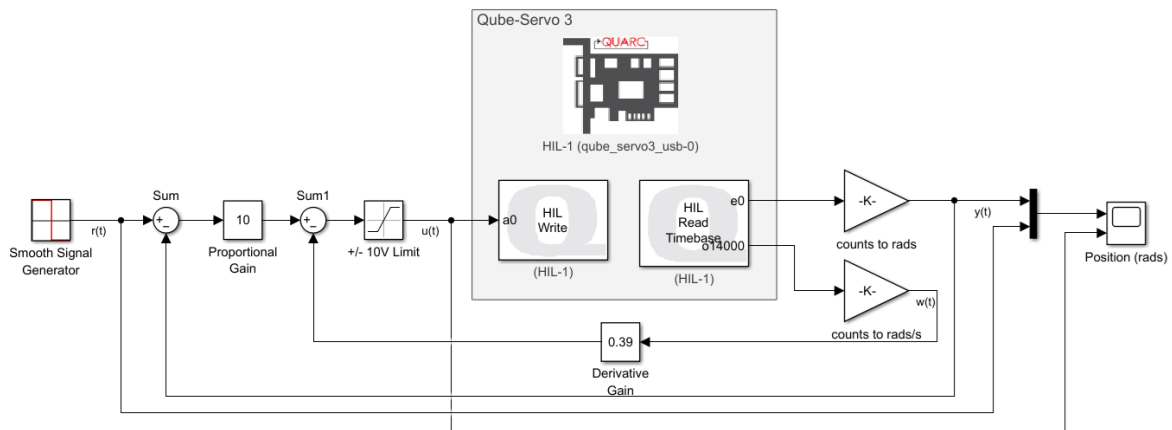


Figura 3.10: Ejemplo de configuración del modelo en Simulink para un controlador Proporcional-Derivativo (PD) en el control de posición del QUBE-Servo 3.

Ejercicio

Ajustar las ganancias proporcional K_p y derivativa K_d para observar cómo afectan a la respuesta del sistema. Se recomienda comenzar con un valor pequeño para K_d y aumentarlo gradualmente mientras se ajusta K_p para obtener una respuesta estable, rápida y sin oscilaciones excesivas.

En la Figura 3.11 se muestra la respuesta del sistema a una entrada de referencia de señal cuadrada de 0.4Hz y $\pi/4$ de amplitud, para un controlador PD con $K_p = 10$ y $K_d = 0.39$. Se puede observar que la respuesta es más rápida y con un error en estado estacionario menor que el controlador proporcional, además de presentar menos oscilaciones. De igual forma en la Figura 3.12 se muestra la respuesta del sistema para un controlador PD con $K_p = 2.5$ y $K_d = 0.05$, donde se puede observar que la respuesta presenta un sobre disparo debido a un valor de ganancia derivativa menor.

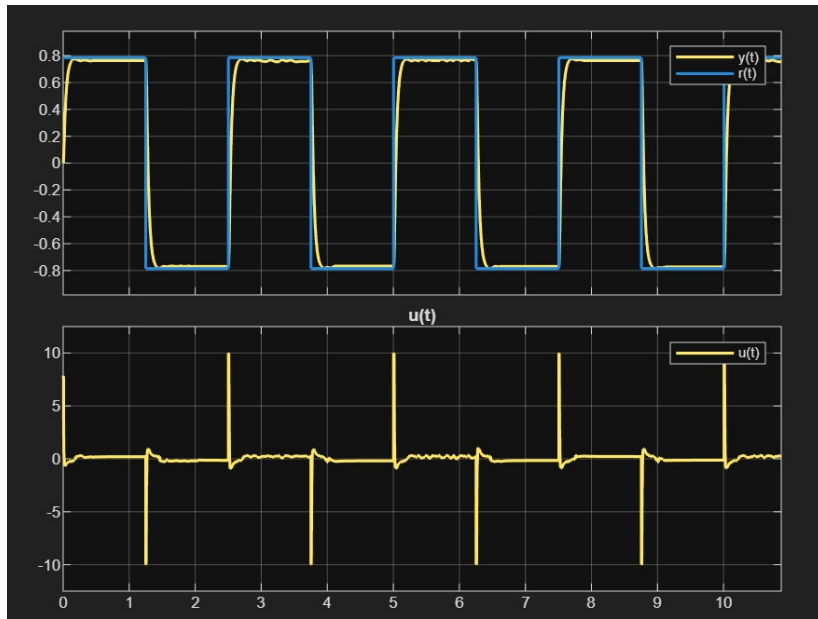


Figura 3.11: Respuesta del sistema con un controlador Proporcional-Derivativo (PD) para $K_p = 10$ y $K_d = 0.39$.

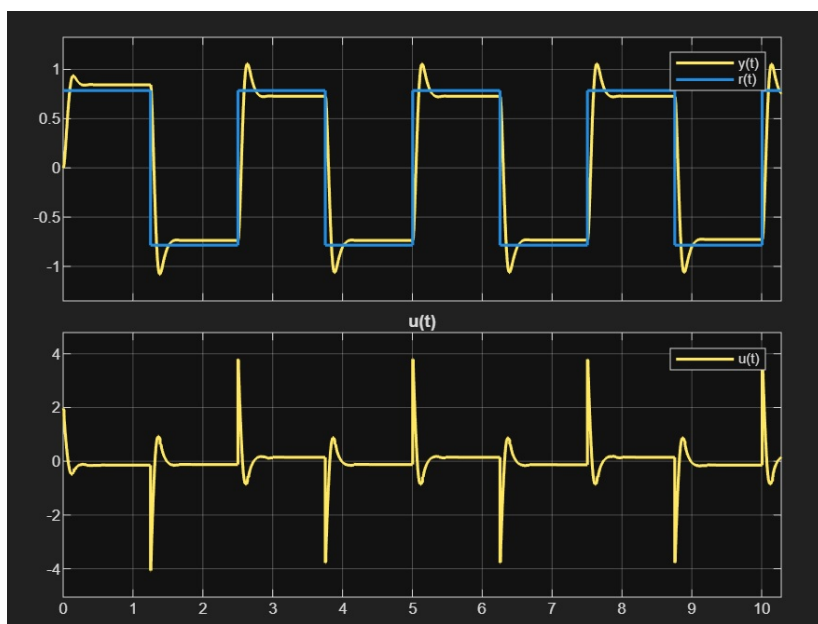


Figura 3.12: Respuesta del sistema con un controlador Proporcional-Derivativo (PD) para $K_p = 2.5$ y $K_d = 0.05$.

3.5 Reto

En esta sección se propone como reto la implementación y sintonización experimental de un controlador Proporcional-Integral-Derivativo (PID) para el control de posición del QUBE-Servo 3.

Modelo del controlador PID para posición

La ley de control ideal en el dominio del tiempo está dada por:

$$u(t) = K_p e(t) + K_i \int_0^t e(\tau) d\tau + K_d \frac{de(t)}{dt} \quad (3.7)$$

donde $e(t) = \theta_d(t) - \theta_m(t)$ es el error entre la referencia angular y la posición medida.

Para esta práctica, se recomienda mantener la misma implementación física usada en el controlador PD, es decir, emplear la velocidad angular medida $\omega_m(t)$ en la rama derivativa. De esta forma, la ley de control práctica puede expresarse como:

$$u(t) = K_p e(t) + K_i \int_0^t e(\tau) d\tau - K_d \omega_m(t) \quad (3.8)$$

La planta de posición puede aproximarse por:

$$G(s) = \frac{\Theta_m(s)}{V_m(s)} = \frac{K}{s(\tau s + 1)} \quad (3.9)$$

Objetivo del reto

Implementar un controlador PID de posición en Simulink y ajustar sus ganancias de forma experimental hasta lograr una respuesta rápida, estable y con error en estado estacionario mínimo para una referencia angular variable.

Procedimiento sugerido

1. Partir del controlador PD previamente implementado en la práctica guiada.
2. Agregar una rama integral a partir de la señal de error $e(t)$, utilizando un bloque integrador y una ganancia ajustable K_i .
3. Ajustar iterativamente K_p y K_d .

Criterios de análisis

Durante la sintonización del PID, evaluar al menos los siguientes aspectos:

- Sobreimpulso.
- Tiempo de establecimiento.
- Rapidez de respuesta.
- Error en estado estacionario.

3.6 Preguntas de análisis

1. Compare la respuesta del sistema con control P, PD y PID. Explique cómo cambia el seguimiento de posición al agregar la acción derivativa y luego la acción integral.
2. Analice el efecto de la acción integral en el controlador PID de posición. ¿Por qué ayuda a reducir el error en estado estacionario? ¿Qué ocurre cuando K_i es demasiado alto?
3. En la implementación práctica del PID, se utiliza la velocidad angular medida en la rama derivativa. Explique por qué esta estrategia es preferible a derivar directamente la señal de error cuando la referencia cambia de forma brusca.
4. Con base en sus pruebas experimentales, describa el efecto individual de cada término (K_p , K_i , K_d) e indique qué sucede cuando:
 - K_p es demasiado bajo o demasiado alto.
 - K_i es muy alto.
 - K_d es muy bajo o muy alto.

3.7 Entregables

Entregables

1. **Demostración experimental:** validación ante el docente del funcionamiento del controlador PID de posición implementado sobre el QUBE-Servo 3.
2. **Informe técnico en formato IEEE**, que incluya:
 - **Implementación del reto:**
 - Estructura del controlador PID implementado en Simulink.
 - Procedimiento de sintonización experimental de K_p , K_i y K_d .
 - **Análisis de resultados:**
 - Gráficas comparativas de respuesta para lazo abierto, control P, control PD y control PID.
 - Discusión y comparación del efecto de cada parámetro en: Tiempo de establecimiento, sobreimpulso, error en estado estacionario.
 - **Respuestas a las preguntas de análisis.**
3. **Anexo de código:**
 - Modelos de Simulink desarrollados
 - Código debidamente comentado y organizado

3.8 Rúbrica de Evaluación

Cuadro 3.1: Rúbrica de evaluación — Práctica: Control de posición con Qube-Servo 3 (10 puntos)

Criterio	Peso	Nivel alto (9–10)	Nivel medio (6–8) / bajo (0–5)
Reto: Controlador PID	70 %	Implementa y sintoniza el PID de forma experimental. Evidencia clara del efecto de K_p , K_i y K_d en la respuesta del sistema.	PID incompleto o mal ajustado. No se evidencia análisis del efecto de las ganancias.
Informe y análisis	20 %	Informe claro, organizado y técnico (formato IEEE). Incluye gráficas, comparaciones y conclusiones coherentes.	Informe desordenado, incompleto o sin análisis suficiente. Falta de claridad en resultados.
Código y simulación	10 %	Modelos de Simulink y scripts correctamente estructurados, comentados y funcionales.	Código desordenado, sin comentarios o con errores de ejecución.

Capítulo 4

Práctica 3: Controlador de posición para helicóptero de un grado de libertad

Modalidad: Presencial

Valoración: 10

Grupos: Máx. 2 o 3 estudiantes

Entrega: Según calendario

4.1 Introducción

El control de sistemas dinámicos es una de las bases fundamentales en el desarrollo de sistemas robóticos, donde es necesario regular variables físicas como posición, velocidad o fuerza mediante el uso de actuadores y sensores.

En esta práctica se aborda el problema de control de un helicóptero de un grado de libertad, el cual constituye un sistema dinámico no lineal. Este tipo de sistemas es representativo en robótica debido a la interacción entre dinámica mecánica, electrónica de control y procesamiento de señales.

Se adopta un enfoque experimental, donde el comportamiento del sistema se obtiene a partir de mediciones, sin requerir el conocimiento exacto de sus parámetros.

4.2 Objetivos de aprendizaje

Objetivos de Aprendizaje

Objetivo general

Desarrollar e implementar un sistema de control PID para regular la posición angular de un helicóptero de un grado de libertad, a partir de su comportamiento dinámico experimental.

Objetivos específicos

- OE1.** Implementar experimentalmente un controlador PID para una zona lineal del sistema.
- OE2.** Diseñar e implementar un controlador PID con ganancia programable para regular la posición angular.
- OE3.** Evaluar el desempeño del sistema de control bajo diferentes referencias.

4.3 Materiales requeridos

Materiales y Equipos

- Helicóptero de un grado de libertad
- ESP32 S3
- Cable USB C
- Fuente de alimentación 12V DC 20A.
- Python (para implementar el control) y Arduino IDE (para programación del ESP32)

4.4 Procedimiento

El desarrollo de la práctica se divide en dos fases: una etapa guiada para comprender las limitaciones del control lineal y un reto el cual será desarrollado por los estudiantes.

Nota

- Antes de comenzar, asegúrese de que el entorno de desarrollo esté correctamente configurado y que el helicóptero y ESP32 estén desenergizados.
- El material de referencia para esta práctica se encuentra disponible en el [repositorio del curso](#) .

4.4.1 Controlador PID en zona local

En esta fase se validará la comunicación con el hardware y se estabilizará el sistema en un único punto de operación.

P1. Interacción Serial Directa: Antes de ejecutar cualquier script, conecte el ESP32 S3 y abra el *Serial Monitor* del Arduino IDE (configurado a 115200 baudios).

- Envíe el comando **CAL** y espere la confirmación sonora del ESC.
- Pruebe manualmente diferentes niveles de potencia con **M:<pct>** (ej. **M:40**) y observe la respuesta del helicóptero.
- Use el comando de emergencia **I** para verificar el retorno inmediato al estado **IDLE**.

P2. Control Manual desde Python: Ejecute el script `o_Serial_Com.py`. Este programa permite enviar comandos mientras guarda la telemetría en tiempo real dentro de la carpeta **Results/**.

- Seleccione el puerto COM correspondiente al ESP32 S3.
- Ingrese el comando **help** para visualizar las opciones disponibles.
- Ingrese la siguiente secuencia de comandos para probar la resolución del sistema: **M:10.1**, luego **I**, y finalmente **M:15.0**.
- Ingrese el comando **q** para finalizar la sesión de captura de forma segura.

Para revisar los resultados de su experimento, ejecute el script: `o_Results_Serial_Com.py`. Analice las gráficas de Ángulo y PWM generadas.

P3. Linealidad del sistema: Ejecute el script `1_Steps_Response.py`, espere a realizar el experimento y luego revise los resultados. Observe cómo la respuesta del sistema varía al aplicar diferentes escalones de referencia (ej. 5° , 20° , 45°) y cómo se manifiestan las no linealidades en cada caso.

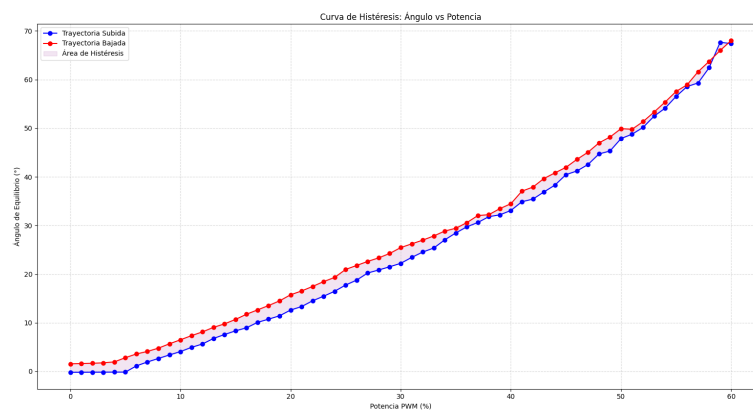


Figura 4.1: Respuesta a escalones de referencia con pasos de 1 en 1. Se observa que la dinámica del sistema no es completamente lineal.

P4. Sintonía de PID en Zona Local (15°): Se aplicará el método del relé con histéresis (Aström-Hägglund) para identificar la ganancia crítica y el periodo de oscilación en un punto de operación específico, lo que permitirá calcular las constantes del controlador PID.

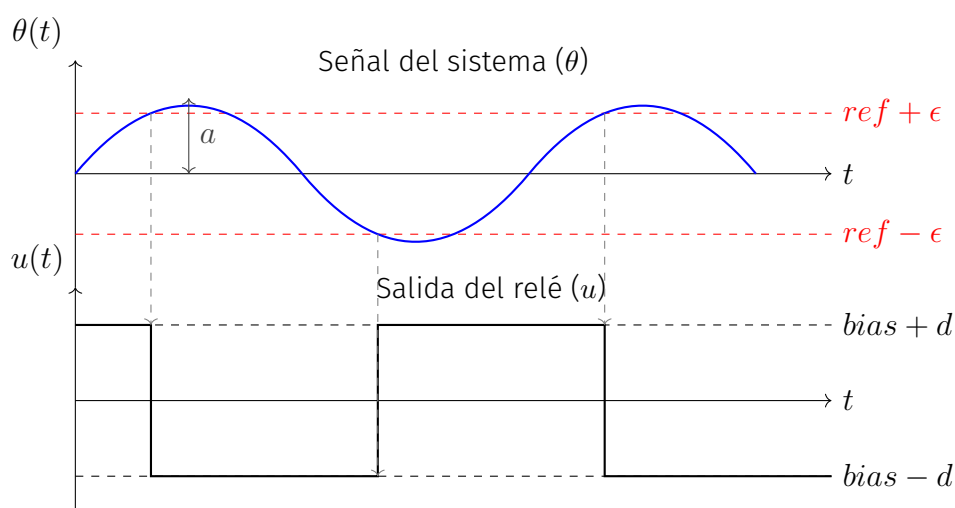


Figura 4.2: Dinámica del método del relé con histéresis. La señal de control $u(t)$ conmuta entre $bias \pm d$ cuando la variable de proceso $\theta(t)$ cruza los límites impuestos por la semi-banda de histéresis ϵ alrededor de la referencia.

- **Configuración Inicial:** Ejecute el programa `2_Rele.py` e ingrese los siguientes parámetros iniciales a través de la consola:

- **Referencia (ref):** 15° (punto de operación local).
- **Bias:** 0.0% (el motor iniciará en estado de reposo).
- **Amplitud del relé (d):** 0.1% (valor mínimo de seguridad).
- **Histéresis (ε):** 3.0° (banda para filtrar ruido inicial).

• **Procedimiento Experimental:**

- (a) Definir la referencia mediante el comando `ref <valor>` (ej. `ref 15`).
- (b) Definir el rango de histéresis con `eps <valor>` (ej. `eps 3`).
- (c) Incremente gradualmente el valor de `bias` mediante el comando `bias <valor>` hasta que el brazo del helicóptero se eleve y se estabilice cerca de la referencia programada.
- (d) Incremente gradualmente la potencia mediante el comando `d <valor>` (se sugiere un valor entre 3% y 7%) hasta provocar una **oscilación sostenida** y simétrica.
- (e) Una vez lograda la oscilación estable (ciclo límite), capture datos durante al menos 10 segundos y presione la tecla `q` para finalizar el experimento y generar el archivo `rele_muestreo.csv`.

- **Análisis de Resultados:** Ejecute el script de procesamiento de datos para visualizar la respuesta. A partir de la gráfica generada, identifique la amplitud de la oscilación de salida (a) —medida como el valor pico desde la referencia— y el periodo crítico o de oscilación (T_u).

Calcule la ganancia crítica (K_u) utilizando la amplitud del relé (d) y la semi-banda de histéresis (ϵ) mediante la función descriptiva para un relé no ideal:

$$K_u = \frac{4 \cdot d}{\pi \sqrt{a^2 - \epsilon^2}} \quad (4.1)$$

- **Cálculo de Constantes PID:** Utilice las reglas de sintonía de la Tabla 4.1 para determinar los parámetros del controlador PID a partir de los valores obtenidos (K_u y T_u).

Cuadro 4.1: Sintonía de PID según Aström-Hägglund / Ziegler-Nichols.

Controlador	K_p	T_i	T_d
PID	$0.6K_u$	$T_u/2$	$T_u/8$

Para la implementación en el script de Python, convierta los parámetros anteriores a ganancias de la forma paralela utilizando las siguientes relaciones:

$$K_p = K_p, \quad K_i = \frac{K_p}{T_i}, \quad K_d = K_p \cdot T_d \quad (4.2)$$

- **Implementación del controlador PID:** Ejecute el script `3_PID_Control.py` e ingrese los valores calculados para K_p , K_i y K_d .

Este programa implementa la ley de control PID en su **forma incremental**.

La señal de control $u(k)$ se actualiza en cada instante k según la ecuación:

$$u(k) = u(k-1) + \Delta P(k) + \Delta I(k) + \Delta D(k) \quad (4.3)$$

Donde los incrementos se definen mediante la discretización de Euler como:

- $\Delta P(k) = K_p[e(k) - e(k-1)]$
- $\Delta I(k) = K_i \Delta t e(k)$
- $\Delta D(k) = \frac{K_d}{\Delta t} [e(k) - 2e(k-1) + e(k-2)]$

Donde $e(k)$ es el error actual, $e(k-1)$ y $e(k-2)$ son los errores de las muestras anteriores, y $\Delta t = 0.01$ s.

Verifique que la respuesta del sistema sea estable. En caso de oscilaciones excesivas, realice un ajuste fino reduciendo levemente K_p o incrementando la acción derivativa K_d . Al finalizar, presione **q** para almacenar los datos en la carpeta **Results**.

4.5 Reto: Control PID por ganancia programable (*Gain Scheduling*)

El helicóptero 1-DOF presenta una dinámica no lineal debido a efectos gravitatorios y aerodinámicos dependientes del ángulo. El objetivo de este reto es diseñar un esquema de *Gain Scheduling* mediante PID en múltiples puntos de operación.

Instrucciones del Reto

- R1. Sintonía en zona de alta exigencia (Obligatorio):** Repita el proceso de sintonización mediante el método del relé para una referencia de 50° . Obtenga los nuevos valores de K_u , T_u y calcule las constantes K_p , K_i , K_d .
- R2. Validación Cruzada:** Pruebe el controlador diseñado para 15° en la referencia de 50° , y viceversa. Documente el comportamiento (oscilaciones, tiempo de respuesta o error estacionario).
- R3. Opcional: Implementación de *Gain Scheduling*:** Como actividad adicional, caracterice el sistema en puntos intermedios (20° , 30° , 40°) y ajuste automáticamente sus ganancias según la referencia:
- **Enfoque por Tramos (*Switching*):** Switching de ganancias según el rango de θ_{ref} .
 - **Enfoque Continuo:** Utilice una regresión para encontrar una función $K(\theta_{ref})$ que aproxime la relación entre el ángulo y las ganancias.
- R4. Validación del Desempeño Dinámico:** Ejecutar el experimento y enviar referencias que varíen entre 0° y 45° para evaluar la capacidad del sistema de mantener la estabilidad y el seguimiento preciso en el rango dado.

4.6 Preguntas de análisis

- P1.** ¿Es posible lograr un desempeño óptimo en todo el rango (0° a 65°) usando un único PID fijo? Justifique su respuesta.
- P2.** Si el periodo de muestreo fuera mucho más lento ($\Delta t = 0.5$ s), ¿Podría el controlador corregir el error antes de que el brazo sobrepase la referencia? Relacione su respuesta con la rapidez de movimiento observada físicamente.
- P3.** En relación al controlador PID: ¿Qué efecto tienen K_p ante un error grande? ¿Qué logra K_i cuando el brazo se estabiliza por debajo de la referencia (error de estado estable)? ¿Qué cambio nota en la estabilidad o "frenado" del brazo al aumentar K_d ?

4.7 Entregables

Entregables

1. **Demostración de funcionamiento:** Validación ante el docente del seguimiento de referencias y estabilidad del helicóptero 1-DOF en tiempo real.
2. **Informe técnico en formato IEEE** que incluya:
 - **Reto de Gain Scheduling:**
 - Procedimiento de caracterización en al menos 2 puntos.
 - Tabla comparativa de las ganancias obtenidas en los 6 puntos de operación definidos.
 - Explicación del modelo de transición implementado (regresión polinómica o lógica de conmutación).
 - Gráfica de respuesta del sistema a una referencia variable.
 - **Responder preguntas de análisis.**
3. **Anexo de Código:** Scripts de Python finales, debidamente comentados y organizados.

4.8 Rúbrica de Evaluación

Cuadro 4.2: Rúbrica de evaluación — Práctica: Control de Helicóptero 1-DOF (10 puntos)

Criterio	Peso	Nivel alto (9–10)	Nivel medio (6–8) / bajo (0–5)
Reto: PID con ganancia programable	70 %	Diseña e implementa una transición efectiva de ganancias. El sistema es estable y preciso en el rango operativo.	Implementación parcial o inestable. El sistema no logra compensar el torque gravitacional en puntos altos.
Informe y Análisis	20 %	Documentación impecable bajo formato IEEE. Análisis crítico de la no linealidad y respuestas coherentes.	Informe desorganizado, falta de gráficas de telemetría o análisis conceptual insuficiente.
Código y Discretización	10 %	Código comentado, modular y sin errores de ejecución. Implementa correctamente la discretización de Euler.	Código desordenado, falta de comentarios o errores en la lógica de muestreo (Δt).

Bibliografía

- [1] Quanser Inc., *QUBE-Servo 3*, <https://www.quanser.com/products/qube-servo-3/>, Accessed: 2026-04-15, 2026.
- [2] Quanser Inc., *QUBE-Servo 3 Experiment: User Manual – Setup and Configuration*, Version 1.0, Markham, Ontario, Canada, 2023. dirección: <https://www.quanser.com>
- [3] G. F. Guzmán Guillén, «Implementación de un controlador predictivo basado en modelo con restricciones sobre un microcontrolador de gama alta,» Tesis de maestría, 2018.
- [4] O. KATSUHIKO, *Teoría de control moderna*, 2003.